MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Analysis of transaction data from CICS and MQ systems

Bachelor's Thesis

VLADIMÍR BOREK

Brno, Spring 2025

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Analysis of transaction data from CICS and MQ systems

Bachelor's Thesis

VLADIMÍR BOREK

Advisor: Mgr. Zuzana Schwarzová, Consultant: Ing. Aleš Malý

Department of Computer Systems and Communications

Brno, Spring 2025

FACULTAS ARTIS INFORMATICÆ • UNIVERSITAS MASARYKIANA •

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, I used the following AI tools:

- ChatGPT for writing improvements, assistance with the development of the CICS data parser, assistance with generation of related graphs, support with BibTeX citation formatting and support with finding some sources.
- Grammarly for grammar check,

I declare that I used these tools in accordance with the principles of academic integrity. I checked the content and took full responsibility for it.

Vladimír Borek

Advisor: Mgr. Zuzana Schwarzová, Consultant: Ing. Aleš Malý

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my consultant, Ing. Aleš Malý, for his exceptional guidance, continuous support, and the many hours he dedicated to our consultations. His expertise, insightful feedback, and deep knowledge of mainframe systems were instrumental throughout the development of this thesis.

I would also like to thank my advisor, Mgr. Zuzana Schwarzová, for her thoughtful feedback on the thesis and for dedicating her time and attention despite her demanding schedule. Her willingness to make time for consultations significantly contributed to the refinement and clarity of this work.

Furthermore, I would like to extend my appreciation to Kyndryl for providing the datasets used in this thesis. Their cooperation enabled the practical component of this research and significantly contributed to its relevance.

Lastly, I would like to thank my family and friends for their unwavering encouragement and support throughout my studies.

Abstract

IBM mainframes generate large volumes of performance data through the System Management Facility (SMF) framework, yet extracting meaningful insights from this data remains challenging due to its complexity and limited tool accessibility. This thesis focuses on analyzing SMF records from the Customer Information Control Systems (CICS) and IBM Message Queueing (MQ) subsystems and proposes reusable guidelines for their interpretation, even without commercial analysis tools.

The work combines theoretical background with practical data processing, including the development of a parser for CICS SMF 110 records and a structured approach to performance analysis. Concepts such as the transaction lifecycle and message flow are used to contextualize system behavior. Due to limitations in the available dataset, the MQ-related analysis is based primarily on theory.

The thesis contributes a set of actionable approaches for SMF data analysis and aims to support both newcomers and those experienced with mainframes in understanding and analysing CICS and MQ system performance.

Keywords

SMF, IBM, CICS, MQ, mainframe, transaction, message, performance, life cycle, guidelines

Contents

Acronyms **1**

Introduction **4**

1 Technical Foundations and System Context **6**
1.1 Introduction to Mainframes 6
1.1.1 Mainframes 6
1.1.2 Mainframe Architecture and Core Components 7
1.1.3 The Role of Mainframes and Their Importance . 9
1.2 SMF, CICS, and MQ Fundamentals 10
1.2.1 SMF Overview 10
1.2.2 Deep Dive into CICS 11
1.2.3 Deep Dive into IBM MQ 21

2 Working with SMF Data in Practice **27**
2.1 SMF Data Storage on Mainframes 27
2.2 SMF Data Acquisition 28
2.2.1 CICS SMF 110 Data Processing 28
2.2.2 MQ SMF Data Processing 28
2.3 Records Structure 30
2.3.1 CICS Records Structure 30
2.3.2 MQ Records Structure 33
2.4 Data Preparation for Analysis 35
2.4.1 CICS 35
2.4.2 MQ 35

3 SMF CICS Data Analysis **36**
3.1 SMF CICS 110 Records 36
3.1.1 SMF 110 Data Classes 37
3.1.2 SMF 110 Record Subtypes 38
3.2 Exploratory Dataset Analysis 39
3.3 Guidelines for Analysis 41
3.3.1 Entry Point Contribution Scan 41
3.3.2 Transaction Breakdown 42
3.3.3 Time-Series Correlation 43
3.3.4 Standard Follow up checklist 44

3.4 Practical Demonstration of Guidelines 45
3.4.1 Contribution Scan 45
3.4.2 DTAX Transaction Breakdown 46
3.4.3 Time-Series Correlation 47
3.4.4 Conclusion of the DTAX Analysis 51

4 SMF MQ Data Analysis 52
4.1 Introduction to MQ Records 52
4.1.1 SMF MQ 115 Statistics Records 52
4.1.2 SMF MQ 116 Accounting Records 54
4.2 Interpreting and Deriving Value from MQ Records . . . 56
4.2.1 SMF MQ 115 Statistics Records 56
4.2.2 SMF MQ 116 Accounting Records 61
4.3 Analysis of Available Dataset 65

Bibliography 68

A An appendix 83
A.1 SMF CICS 110 Parser Overview 83
A.2 Formatted Datasets Used in Analysis 83
A.3 JCL Scripts for SMF Data Formatting 84
A.4 Figures Folder 84

List of Figures

1.1 Transaction Life Cycle in a CICS Region [51]. 18
1.2 Response Time Decomposition[52, p. 76] 20
1.3 Simple configuration of MQ Queue Managers. [67] . . . 24
1.4 High-level IBM MQ architecture overview showing message, buffers, buffer pool, and page sets. [68] 25

2.1 Structure of the Metadata Section in a CICS SMF Record . 30
2.2 Segment of a performance class CICS transaction from the available dataset `STUDENT.CICS` 31
2.3 Partial hex view of an SMF 116 record, illustrating the header and the self-definig section. Source: [76] 33
2.4 Segment of buffer manager statistics record from the available dataset `STUDENT.MQS`. 34

3.1 Distribution of transactions across all days and regions. . 40
3.2 Contribution scan of transactions across performance metrics. 45
3.3 suspend time across all Regions on 2025-02-19. [1] 48
3.4 Dispatch time across all Regions on 2025-02-19. [1] 48
3.5 RMITIME across all Regions on 2025-02-19. 49
3.6 SUSPTIME across all regions on 2025-02-18. [1] 50
3.7 Dispatch Time supplemented with CPU-time and SYNCTIME submetrics across all regions on 2025-02-18. [1] 50

4.1 MQ request activity across time intervals. [114] 58
4.2 Buffer pool usage across subsystems. Source: [114] 60
4.3 Health insights—drilldown by buffer pool. Source: [114] 61
4.4 MQ CPU usage by connection type and command [126] . 62
4.5 Breakdown of time spent in MQ for transaction AAAA [118] 63
4.6 MQGET call rate vs. elapsed time for CKTI transaction [126] 64

Acronyms

APIs Application Programming Interfaces

BDAM Basic Direct Access Method

BMS Basic Mapping Support

BTAM Basic Telecommunications Access Method

CECI CICS Execute Command Interpreter

CEDF CICS Execute Debug Facility

CF Coupling Facility

CICS Customer Information Control System

CP Central Processor

CPC Central Processor Complex

DASD Direct Acess Storage Devices

DWP Deferred Write Process

EMPs Event Monitoring Points

FCP File Control Program

FCT File Control Table

IBM International Business Machines Corporation

I/O Input/Output

ISC Inter-System Communication

JCL Job Control Language

KCP Task Control Program

LPAR Logical Partitioning

MCA Message Channel Agent

MQ IBM Message Queuing

MQMD MQ Message Descriptor

MRO Multi-Region Operation

PCP Program Control Program

PCT Program Control Table

PDS Partitioned Data Sets

PPT Processing Program Table

QM Queue Manager

QR TCB Quasi-Reentrant Task Control Block

RAS Reliability, Availability, and Security

RMF Resource Measurement Facility

RMI Resource Manager Interface

SAP System Assistance Processor

SCP Storage Control Program

SMDS Shared Message Data Set

SMF System Management Facility

SOS Short on Storage

SYSTEM-ID System Identifier

TCB Task Control Block

TCP Terminal Control Program

TCT Terminal Control Table

VSAM Virtual Storage Access Method

VTAM Virtual Telecommunications Access Method

WLM Workload Manager

zIIP z Integrated Information Processor

Introduction

One of the key strengths of mainframes, enterprise-grade computing systems, is their ability to produce an incredibly rich set of performance and diagnostic metrics through the System Management Facilities (SMF) framework. However, this also introduces challenges. Collecting large volumes of SMF data can strain system resources, potentially affecting performance. [1]

Moreover, based on conversations with the consultant of this thesis, who has long-term experience with mainframes, obtaining and interpreting SMF data is often non-trivial. Access to the data is restricted, specialized tools are required, and commercial analysis solutions are not always available or affordable. This thesis aims to highlight the potential of learning from SMF mainframe data, particularly from CICS and MQ subsystems, by demonstrating how meaningful insights can be extracted even without premium tooling.

The main objective of this thesis is to analyze SMF records from transactional systems on the IBM z/OS platform and to develop guidelines for their effective interpretation, with the goal of maximizing the impact on system operation and end users. In doing so, it also aims to provide insight into the operational behavior of mainframes and their subsystems, specifically CICS and MQ, through practical and structural insights derived from performance data. The thesis describes the transaction lifecycle, explains the structure of relevant SMF data, and formulates reusable analytical patterns that can support performance tuning, system monitoring, or general learning. The results aim to benefit both newcomers who are learning about mainframes and experienced professionals seeking a deeper understanding of CICS and MQ behavior. The guidelines and interpretations are intended to be flexible enough to apply in various environments, with or without the assistance of specialized commercial tools.

The structure of the thesis is organized as follows:

Chapter 1 introduces the foundational technical concepts, including mainframe architecture and key subsystems such as SMF, CICS, and MQ. It also explains core ideas like transaction lifecycle and message flow, building the context necessary for understanding the analyzed data.

Chapter 2 presents a practical overview of how SMF data is acquired, stored, and prepared for analysis. It describes the structure of CICS and MQ SMF records and discusses data preparation strategies, emphasizing the necessity of parsing data into a structured format.

Chapter 3 focuses on the analysis of SMF 110 records generated by CICS. It includes an exploratory dataset overview and introduces a set of structured analysis guidelines, derived from both theoretical understanding and hands-on experience. The chapter concludes with a detailed analysis of a selected transaction, demonstrating how the guidelines translate into actionable insights.

Chapter 4 analyzes SMF 115 and 116 records produced by IBM MQ. It explains their purpose, outlines how value can be derived from them, and reflects on the limitations of the available dataset. Unlike the CICS analysis, this chapter is based on theoretical understanding rather than hands-on work.

1 Technical Foundations and System Context

This chapter introduces the core technical concepts necessary to understand the mainframe, CICS, and MQ environments.

It begins with an overview of IBM mainframes, their architecture and components, and the roles they play in enterprise systems.

Next, the focus moves on to the essential subsystems relevant to this thesis: SMF, CICS, and IBM MQ. It explains their role within the mainframe ecosystem and introduces the core concepts, components, and architectural principles necessary to understand how these subsystems function. The chapter outlines the basic principles behind transactions in CICS and message handling in MQ, along with the internal mechanisms and supporting infrastructure that enable them.

These concepts are then tied together by describing the transaction life cycle in CICS and the message flow within MQ, illustrating how the previously introduced mechanisms operate in practice and interact within their respective subsystems.

1.1 Introduction to Mainframes

This section introduces the mainframe computing environment, with a focus on IBM z/OS systems. It outlines the core architectural components, operating systems, and subsystems that define mainframe functionality and highlights their role in supporting high-volume, mission-critical workloads across enterprise environments.

1.1.1 Mainframes

Mainframes are enterprise-grade computing systems designed for processing large volumes of data and executing high number of transactions in real time. They are designed to meet strict demands for reliability, availability, and security (RAS). [2]

While there are various mainframe systems, IBM has become the de facto industry standard. Since this thesis is conducted in collaboration with Kyndryl, and the datasets it works with come from IBM z/OS mainframe, all references to mainframes, their systems, components, and concepts throughout this work specifically refer to those operating

on or in relation to IBM z/OS. While some references may draw from other mainframe environments or operating systems, the underlying concepts and architectural principles remain broadly applicable and relevant across mainframe platforms.

1.1.2 Mainframe Architecture and Core Components

Hardware Layer

At the hardware level, the core of a mainframe is the Central Processor Complex (CPC), that houses multiple types of specialized processors. Each processor type serves a distinct role in optimizing performance, managing resource allocation, and reducing software licensing costs. For example, the standard Central Processor (CP) is responsible for running general-purpose workloads, including the operating system and application-level software. In addition, every mainframe includes at least one System Assistance Processor (SAP), which is dedicated to managing I/O subsystem operations. Another specialized processor is the z Integrated Information Processor (zIIP), which is optimized for database-related workloads, particularly those executed by IBM DB2. It acts as a data hub, enabling efficient access to structured data while contributing to cost optimization. [3]

In addition to the processor complex, mainframes are equipped with large amounts of main memory (even up to 40 terabytes [4]). Specialized I/O channels manage high-speed data transfer between the processors and external devices [5], such as Direct Access Storage Devices (DASD [6]) for disk storage and communication interfaces for interacting with external networks [7].

Operating Systems

To ensure workload isolation, control over resource allocation and improvement in system utilization, mainframes support logical partitioning (LPAR), a form of hardware-based virtualization. This allows a single physical mainframe to be divided into multiple independent virtual environments. Each of these environments is capable of running its own operating system instance. [8]

z/OS is a secure and scalable environment designed to manage batch processing, transaction workloads and networked applications,

while ensuring high availability, resource management and backward compatibility .[9]

While this thesis focuses on the z/OS, it is worth noting that IBM mainframes can support multiple operating systems concurrently through logical partitioning.

Common alternatives include z/VM (a hypervisor for running virtual machines), z/VSE (used in smaller environments), Linux for System z, and z/TPF (optimized for high-volume transaction processing). Among these, z/OS is IBM's flagship operating system and is thus the most widely used in enterprise environments. [10, 11]

Subsystems

Above the hardware layer, mainframes rely on a stack of subsystems. These subsystems are collections of programs that provide specialized services to the operating system and the workloads it manages. Often classified as middleware, subsystems extend core system functionality by offering capabilities not provided directly by the operating system. They typically include application programming interfaces (APIs). Examples of key subsystems include [12, 13, 14]:

- **CICS** (**Customer Information Control System**) Responsible for online transaction processing.
- **DB2** A high performance relational database management system designed for large scale processing.
- **IBM MQ** Enables asynchronous messaging between distributed applications.

Jobs

In the context of z/OS, a job represents a complete unit of work submitted for processing by the system. Jobs are typically defined using Job Control Language (JCL) and consist of multiple programs, utilities, and system operations bundled together. Jobs run in their own address spaces and are scheduled by workload managers (WLM), which distribute workload resources in the system. [15]

Tasks

Within each address space created for a job or system service, multiple tasks may be initiated. A task is a smaller unit of work that operates within the broader job structure. (Further explained in Task.)

Address Space

In the z/OS operating system, an address space refers to the range of contiguous virtual addresses allocated to a user or program for executing instructions and managing data. Each address space encapsulates both system-level code and data, as well as user-level code and data, providing a controlled and isolated environment for task execution. Within a single address space, multiple tasks can be initiated and run concurrently. z/OS employs a large number of distinct address spaces to manage its workloads effectively. For instance, a separate address space is created for each active job and for every user session currently logged in. Additionally, each major subsystem—such as CICS—operates within its own dedicated address space. Beyond application-specific spaces, z/OS also maintains address spaces for various operating system functions, such as operator communication, automation, networking, and security functions. [16]

1.1.3 The Role of Mainframes and Their Importance

Mainframes offer a combination of high throughput, strong security, scalability, and long-term backward compatibility, supported by core RAS design principles. These qualities have contributed to the stable and enduring role of mainframes within the IT landscape. They form the backbone of critical infrastructure across sectors such as finance, healthcare, government, and large-scale enterprise environments. Mainframes are extensively used by largest banks, airlines, insurance providers, and other organizations that require continuous high-volume processing and consistent system stability. Their role in these contexts reflects not only a legacy of long-term adoption but also the persistent demand for dependable and scalable computing solutions. [2]

1.2 SMF, CICS, and MQ Fundamentals

This section establishes the technical foundations required to understand the mainframe environment in which the analyzed data was collected. It introduces the architecture, operating principles, and key subsystems of IBM z/OS, with a focus on CICS and MQ as the transactional and messaging components central to this thesis.

1.2.1 SMF Overview

System Management Facility (SMF) is a built-in component of the IBM z/OS operating system that provides a general framework for recording detailed system and job-related operational and performance data. It acts as a centralized hub where this information is stored into standardized SMF records, each identified by a specific record type. These records are either written directly by SMF itself or generated by external applications utilizing the SMF framework. SMF operates continuously in the background, collecting raw data from a wide range of system and application activities, including CPU usage, memory consumption, I/O operations, start and end times of jobs or transactions, and interactions with previously mentioned subsystems such as CICS, MQ, and DB2. The data can be used for various purposes such as billing users, performance tuning, planning future capacity need and auditing system activities. [17, 18, 19]

Record Types

Each SMF record is assigned a specific record type number that defines the kind of data it contains. Some record types are further divided into subtypes to capture more detailed or specialized information. [20]

System administrators have control over which SMF record types are generated and how frequently they are recorded [21]. Some record types also allow configuration of the level of detail recorded. This flexibility is important because capturing excessive amounts of SMF data can lead to measurable CPU overhead (extra CPU work the system has to do) and increased system resource consumption. This can raise operating costs. Therefore it is common practice to enable only the SMF records necessary for monitoring, and to activate additional record

types ad-hoc and temporarily when deeper analysis is required to resolve a specific problem. [22]

Some of the most common relevant record types are:

- **Type 30** - Records common address space activity, including resource usage by jobs, users, and started tasks. It can also be used to monitor file system activity for different users classes, assisting in forecasting DASD and system resource needs. [23]

- **Type 70-79** - Capture system performance metrics collected by the Resource Measurement Facility (RMF), such as CPU utilization, memory usage, cache efficiency, DASD performance, and workload distribution. These records are useful for capacity planning, system performance analysis and workload management. [24, 25]

- **Type 100-101** - Capture DB2 subsystem data.

  Type 100 records subsystem-wide statistics such as buffer pool usage, locking activity, and SQL processing rates, which can provide insights about potential problems. [26, 27].

  Type 101 records per-thread or per-connection accounting information, including resource usage, SQL execution metrics, and transaction timings [28, 29].

- **Type 119** - Records TCP/IP network activity, including session establishment and connection metrics. It is relevant in cases where transaction performance may be affected by network delays. [30]

The SMF record types listed above provide a broad overview of the main performance and system monitoring data available. Among them, the record types 110, 115 and 116, which are described in more detail in Sections 3.1, 4.1.1, and 4.1.2, play a particularly important role in this thesis focused on analysis of transaction data from CICS and MQ systems.

1.2.2 Deep Dive into CICS

This subsection offers an overview of the CICS transaction processing environment, focusing on its architecture, key components, and exe-

cution model. It also introduces the transaction lifecycle and breaks down the transaction's response time and its key components.

Introduction to IBM CICS

CICS (Customer Information Control System) is a general-purpose transaction management subsystem. It is designed to process high volumes of online transactions while ensuring compliance with the principles of atomicity, consistency, isolation, and durability, collectively known as the ACID properties. Common examples of online transactions include ATM deposits or withdrawals, credit card payments, and ticket reservations. These transactions typically involve transferring only a small amount of data per operation but are executed in very large numbers and support a high number of simultaneous users. [31, 32]

Transaction Services and System Abstraction

CICS provides transaction processing services that allow application programs to focus solely on business logic without needing to manage underlying system complexities. Instead of interacting directly with terminals, databases, or devices, application programs issue standardized commands to CICS, which handles the communication with the operating system and system resources. This abstraction ensures that critical aspects such as resource allocation, input/output operations, data integrity, fast response times and user authorization are centrally managed by CICS. By handling concurrency control, prioritization and efficient workload management, CICS effectively acts as a specialized transaction-oriented operating environment within z/OS, maintaining high performance and system stability even under heavy multi-user loads. [33, 34]

CICS application

Is a collection of related programs that are executed under the supervision of CICS, leveraging standardized services for accessing files, system resources, and communication interfaces. These programs work together to carry out a specific business operation, such as processing

a product order or managing payroll calculations. CICS applications are typically triggered by submitting transaction requests. [34]

Transaction

In CICS can refer either to an individual user request or to a general type of operation performed repeatedly across many instances. In the broader IT context, the term transaction typically refers to a unit of recovery, a complete, self-contained operation that can be rolled back or committed. In CICS, this is known as a unit of work. Each transaction is identified by a short, four-character transaction identifier (TRANSID), which directs CICS to the initial program responsible for handling the request. This program then invokes other related application programs needed to complete the transaction. A CICS transaction is essentially a collection of logically related programs within an application. In large-scale systems, complex business processes are often broken down into multiple transactions, each responsible for a specific subtask within the overall workflow. [35, 36, 37]

Task

Is created by CICS each time a transaction is initiated to manage the execution of that particular instance. A task manages all aspects of the transaction's execution, including interaction with system resources, communication with services, and access to protected data. In many ways, a CICS task resembles a lightweight thread assigned to handle one user action. Tasks are short-lived, terminating automatically when the associated transaction completes. [37]

Regions

CICS operates within specialized environments called regions, which are independently managed address spaces. A region acts as a self-contained administrative unit, responsible for managing a set of resources and providing transaction processing services to applications. These services include resource coordination, transaction execution, user session management, and access to files and databases. [38]

Within a CICS region, resources such as transactions, Basic Mapping Support (BMS) maps (which provide an interface between CICS and applications, managing input/output formatting [39]), programs, terminals, communication queues and datasets are managed collectively [40]. The region ensures that user applications can run concurrently and securely without directly negotiating with the underlying operating system for access to system resources. [41]

CICS regions use a pool of multithreaded application servers to handle multiple client requests simultaneously. Each transaction initiated by a user triggers a CICS task, which manages memory, processing, and interactions with system services. Tasks within a region are cooperatively multitasked — they are designed to yield the CPU when waiting, allowing efficient sharing of processor time across thousands of concurrent activities. [38, 42]

Specialized Regions To support different operational needs, CICS regions can be specialized. It is often convenient to assign regions to perform particular roles based on the applications they support. While any CICS region is capable of fulfilling all roles, in some cases it is appropriate to combine multiple roles within a single region. [43] :

- **Transport-owning region** - Responsible for connecting to the communication network. It receives transaction requests and forwards them to an application-owning region for processing.
- **Terminal-owning regions (TOR)** – Manages user sessions and handles communication between terminals and applications. Received transaction requests are passed to an application-owning region.
- **Application-owning regions (AOR)** – Executes business logic and application programs requested by users. It manages program execution and forwards data access requests to a data-owning region.
- **Data-owning region** - Controls access to files and databases.
- **File-owning regions (FOR)** – A specific type of data-owning region that focuses on managing access to files and file systems.

Execution Context and the QR TCB: In CICS, each transaction is executed as a task. To carry out this execution, the task runs on a system structure known as a Task Control Block (TCB), a control element used by z/OS to represent an active task within a region. The standard execution environment for most application logic in CICS is the quasi-reentrant TCB (QR TCB). This TCB ensures that only one user task executes at a time within that context, promoting predictable behavior and protecting shared system resources.

Programs running under the QR TCB follow a programming model called quasi-reentrancy, meaning they are structured to consistently maintain valid internal state between CICS command calls. This model allows the CICS dispatcher, a core system component that routes control to the highest-priority task [44], to pause and resume different tasks safely, facilitating cooperative multitasking and efficient processor sharing across thousands of concurrent transactions.

Understanding this context is essential for understanding 3.4.2 later on in this thesis. [45, 46, 47]

CICS Services

CICS provides a range of system services that collectively support reliable and efficient transaction processing. These services abstract low-level operations from application programs, enabling them to focus purely on business logic. The five major categories are [48]:

- **System Services** – Manage the core functioning of CICS
  - **Task Control**: Oversees multitasking by scheduling processor time among concurrent tasks and prioritizing the most important one.
  - **Program Control**: Handles loading and releasing of application programs, ensuring that even if multiple tasks share the same program, only a single copy resides in memory.
  - **Storage Control**: Manages acquisition and release of storage.
  - **Interval Control**: Provides timer-based services to schedule operations or timeouts.

- **Data Communication Services** – provide the interface between application programs and telecommunications access methods such as Basic Telecommunications Access Method (BTAM) and Virtual Telecommunications Access Method (VTAM) for managing data communication. They abstract terminal hardware interactions through facilities like BBMS and support communication between CICS regions,either within the same system via Multi-Region Operation (MRO) or across systems using Inter-System Communication (ISC).
- **Data Handling Services** – Manage interactions with data access methods such as Virtual Storage Access Method (VSAM) and Basic Direct Access Method (BDAM), and database systems like DB2. They provide commands for dataset management, transaction integrity during concurrent access, and recovery support in case of task or system failures.
- **Application Programming Services** – Provide tools for application development, including the command-level interface, debugging facilities like CICS Execute Debug Facility (CEDF), and interactive command testing with CICS Execute Command Interpreter (CECI).
- **Monitoring Services** – Continuously observe system events within the CICS address space, collecting statistical data for performance tuning and system health analysis.

For a detailed explanation of CICS-specific terms and abbreviations, refer to the [49].

Core Components of CICS

The core of a CICS region, also known as the nucleus, consists of essential system programs, that are loaded into memory at startup. These programs deliver fundamental services such as task scheduling, program management, storage control, and transaction dispatching. They operate based on metadata and mappings defined in a set of user-configurable control tables. Together, the control programs and control tables ensure that CICS operates efficiently and securely under heavy multi-user workloads. [50]

Control Programs [50] :

- **Program Control Program (PCP)** – Responsible for locating, loading, and linking application programs during transaction execution, and managing control transfer between programs.
- **Task Control Program (KCP)** – Is responsible for managing the execution of tasks within the CICS environment. It oversees the simultaneous operation of multiple tasks, coordinating their execution and managing their associated properties. KCP also handles all aspects related to multitasking, ensuring that tasks are scheduled and executed efficiently without conflict.
- **Terminal Control Program (TCP)** – is responsible for handling input from terminals in a CICS environment. It manages the reception of messages sent from terminal devices and ensures that hardware communication requirements are maintained. Upon receiving input, TCP requests CICS to initiate the appropriate task.
- **File Control Program (FCP)** – Provides services to application programs for reading, inserting, updating, and deleting records, maintaining exclusive control over file access to preserve data integrity.
- **Storage Control Program (SCP)** – Handles the allocation and deallocation of storage areas.

Control Tables [50] :

- **File Control Table (FCT)** – Defines all files accessed by CICS applications, including file names, types, and record structures.
- **Terminal Control Table (TCT)** – Manages terminal connections within a region. When a user logs in to a CICS terminal, an entry corresponding to that terminal is created in the TCT. This table maintains the terminal IDs associated with the active CICS region, allowing the system to track connected devices. In conjunction with the TCP, the TCT helps identify and process incoming data from terminals, ensuring accurate communication and task initiation.

- **Program Control Table (PCT)** – Maps transaction identifiers (TRANSIDs) to their corresponding program names or IDs. Each entry in the PCT defines which program should be invoked when a specific transaction is initiated. The TRANSID serves as a unique identifier within the PCT, ensuring that each transaction is correctly routed to its designated application program.

- **Processing Program Table (PPT)** – Tracks application programs and mapsets by storing metadata such as program names, usage counters, language type, storage addresses, and load library locations. It ensures that programs are properly loaded and reused to optimize resource use.

Transaction life cycle

When a transaction is requested in CICS, it goes through a structured lifecycle within the CICS region. Multiple instances of the same transaction may be active concurrently, each executed as a separate task. Understanding this is helpful for recognizing how transaction behavior can impact system performance, and vice versa. [51]

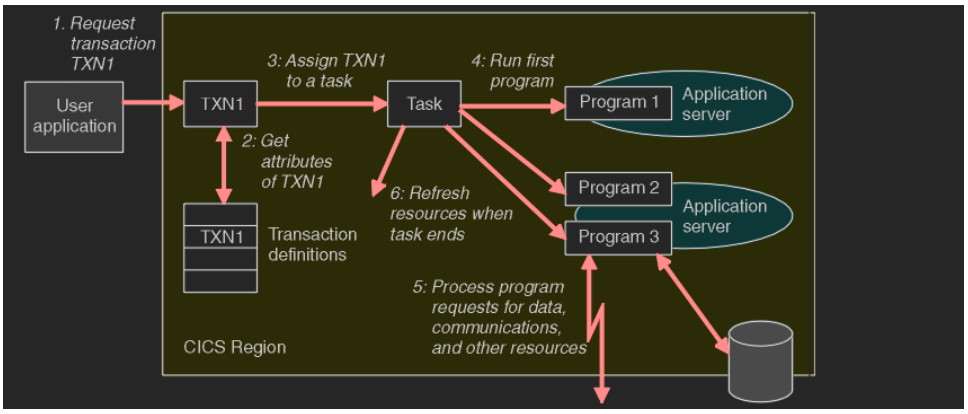


Figure 1.1: Transaction Life Cycle in a CICS Region [51].

The stages of life cycle as shown in Figure 1.1 [51, 50] :

1. Transaction Request and Authorization The CICS region receives a transaction request . This request can originate from various sources,

such as a user terminal, a web service call, an IBM MQ message, an internal CICS program invoking another transaction, or a scheduled job. Upon receiving the request, the CICS region, verifies the communication link and checks authorization to access the system.

2. Transaction Identification and Lookup Next, the CICS region consults its PCT, which maps transaction identifiers (TRANSIDs) to corresponding application programs. It uses the transaction definition to determine essential information, such as the first program to run.

3. Task Creation and Scheduling If the transaction is valid, the CICS region creates a new task to handle its processing. Task management is overseen by the KCP. The PCP first checks the PPT to determine whether the program is already loaded in memory. If not, PCP loads it. The task is scheduled alongside others for processing and access to the required data is acquired.

4. Program Execution Once the task is ready, it begins executing the programs associated with the transaction. The program then runs on a process referred to as an application server, which is essentially part of region that runs business logic. If the transaction requires multiple programs, they may be executed on the same process or distributed across different processes, depending on how they are invoked within the transaction flow.

5. Task Management and Resource Coordination The executing programs might perform computations, database operations, file interactions, or call other services. The CICS region oversees the task's execution, responding to its requests for data, communication, and other system resources. It also performs supporting operations to ensure efficient task progress, protect data integrity, and coordinate properly with other tasks running in parallel.

6. Completion and Cleanup When the transaction finishes, CICS commits any changes to persistent storage. It then terminates the task, frees the allocated resources, and updates monitoring records, such as SMF 110 records.

Transaction Time Components

Response Time A CICS transaction's total duration, from initiation to termination, can be expressed as its response time (also referred to as elapsed time). As illustrated in Figure 1.2, the response time is composed of two main components: suspend time and dispatch time. [52]

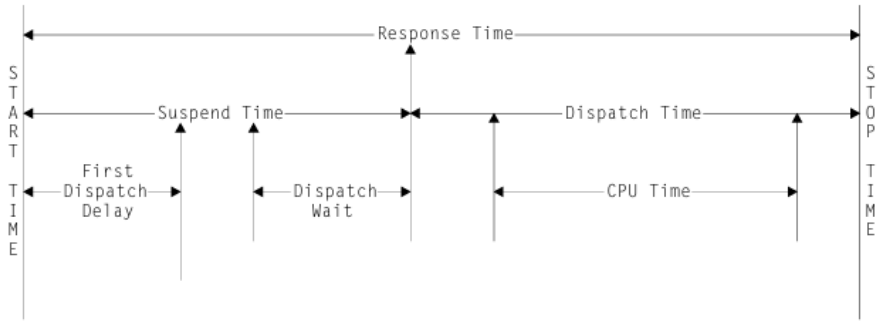


Figure 1.2: Response Time Decomposition[52, p. 76]

Suspend Time represents the interval during which the transaction is not actively executing on the CPU. This typically includes periods of waiting for resources or external operations to complete. suspend time is subdivided into [53, 52, 54] :

- **First Dispatch Delay** – the initial delay before the transaction is first given CPU access.
- **I/O Wait Times** – includes delays due to file access, terminal input/output, temporary storage operations and others.
- **Other (Uncaptured) Wait Times** – includes metrics such as syncpoint delay, lock manager delay, or request receiver wait times.
- **Unclassified Waits** – includes periods of delay not explicitly categorized, often due to shared resource contention or scheduling limitations.

Dispatch Time is the period in which the transaction is eligible to execute and is either executing or actively scheduled on a dispatchable unit. Within this time [53, 52, 54]:

- **CPU Time** accounts for the actual time the transaction runs on the processor, executing its logic.
- **Dispatch Wait Time** covers instances where a transaction is dispatched but temporarily removed from the CPU, without CICS's direct control.

CPU-to-Dispatch Ratio reflects how effectively the transaction utilized its dispatch window. Lower values suggest inefficiencies such as resource contention, poor scheduling, or program design issues, even if they do not directly incur higher cost. Ideally a value of 80% or more is generally desirable. [54]

1.2.3 Deep Dive into IBM MQ

This subsection provides a comprehensive overview of IBM MQ's architecture, terminology, and messaging principles and their workflow overview.

Introduction to IBM MQ

IBM MQ is a high-reliability messaging middleware that facilitates data exchange between diverse applications, systems, and platforms. It enables asynchronous communication, helping decouple components and allowing them to interact independently of time, location, or implementation technology. This makes it especially suitable for enterprise integration across mainframes, cloud environments, and distributed architectures. [55]

Core Terminology and Concepts

Messages and Message Types A message in IBM MQ is a structured sequence of bytes, comprising control metadata contained in the MQ Message Descriptor (MQMD) and application-specific data. It also includes optional message properties such as named fields that carry

business or contextual data. These properties enable efficient filtering and routing using selectors, making it easier to store additional information without embedding it in the message payload. Messages may be persistent (logged and recoverable) or non-persistent (not retained after failure) [56]. Applications can access fields from headers through Message Queue Interface (MQI) calls, which simplifies development and improves message handling. [57, 58]

IBM MQ defines four message types [59]:

- **Datagram:** One-way communication with no expected reply.
- **Request:** Sent when a response from the receiver is needed.
- **Reply:** Sent in response to a request message.
- **Report:** System-generated notifications about events such as errors or delivery confirmations.

Applications typically use datagram, request, and reply messages to exchange data, while report messages are used internally by applications and queue managers to monitor operations.[59]

Queues A queue is a named destination used to hold messages until they are retrieved by an application. Queues allow applications to operate asynchronously, decoupling their execution and availability. Queues may reside in volatile memory or persistent storage, managed entirely by the queue manager. There are multiple types of queues but the most notable are [60, 61, 62] :

- **Local Queue:** A physical queue hosted by the queue manager, where messages are actually stored. Applications can read from or write to these queues directly.
- **Remote Queue Definition:** A logical reference to a queue managed by another queue manager. Remote queues act as forwarding definitions and do not store messages locally.
- **Transmission Queue:** A special type of local queue used as a staging area for messages destined for remote queue managers. Messages are stored here until the associated channel is ready for delivery.

Queue Manager (QM) The queue manager is the central IBM MQ component responsible for managing queues, routing messages, and ensuring once-only delivery even across systems. It interacts with applications via the MQI, handling calls such as `MQOPEN`, `MQPUT`, `MQGET`, `MQCLOSE`, and `MQCOMMIT`. These operations respectively open queues, send messages, retrieve messages, close queues, and commit transactional message operations. Key resources managed by the queue manager include [63, 64, 65] :

- Queue and message storage using page sets and buffer pools
- Logging infrastructure for failure recovery
- Communication links (channels) and messaging protocols
- Interfaces to environments like CICS, IMS, and batch systems on z/OS

Messaging Models IBM MQ supports two primary messaging models [55] :

- **Point-to-point:** Messages are sent to a queue and received by a single consumer. The sender knows the queue name, but not the location or identity of the receiver.
- **Publish/Subscribe:** Messages are published to a topic and delivered to all subscribed consumers. Publishers are unaware of the number or identity of subscribers.

Channels and MQ Networks A channel is a defined logical communication path between queue managers for messages or between an application and a queue manager for MQI commands. Channels abstract the communication protocol, providing consistent and secure message transport over a network. Each end of a channel is managed by a Message Channel Agent (MCA)—a system component that handles message flow. The sender-side MCA retrieves messages from the transmission queue and transmits them across the network. The receiver-side MCA receives these messages and delivers them to the destination queue manager, which stores them in the appropriate local queue. [65, 66]

MQ networks are collections of interconnected queue managers enabling message exchange across systems where applications are connected to different queue managers. [55]

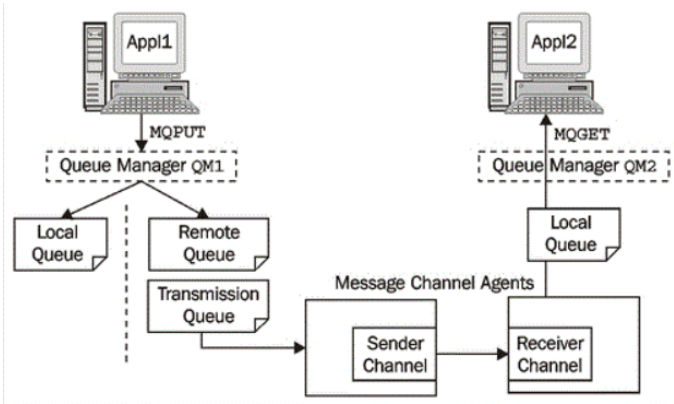


Figure 1.3: Simple configuration of MQ Queue Managers. [67]

Overview of Simple MQ Queue Managers Configuration Figure 1.3 illustrates a basic point-to-point messaging setup in, where two applications communicate through separate queue managers (QM1 and QM2). Application 1 issues an MQPUT call to send a message to a remote queue defined on QM1, which references a local queue on QM2. The message might be temporarily stored in a transmission queue, a special local queue used to hold outbound messages. A sender channel, managed by a MCA, transfers the message across the network to the receiving system. There, a receiver channel MCA accepts the message and delivers it to the target local queue on QM2, from which Application 2 retrieves it using MQGET.

Buffers, Buffer Pools, and Page Sets IBM MQ uses buffers to cache messages temporarily in memory, in order to reduce the performance cost of writing data to and reading data from page sets. Buffers are grouped into buffer pools, each linked to one or more page sets, which are formatted VSAM datasets used for persistent storage of messages and other objects. [68]

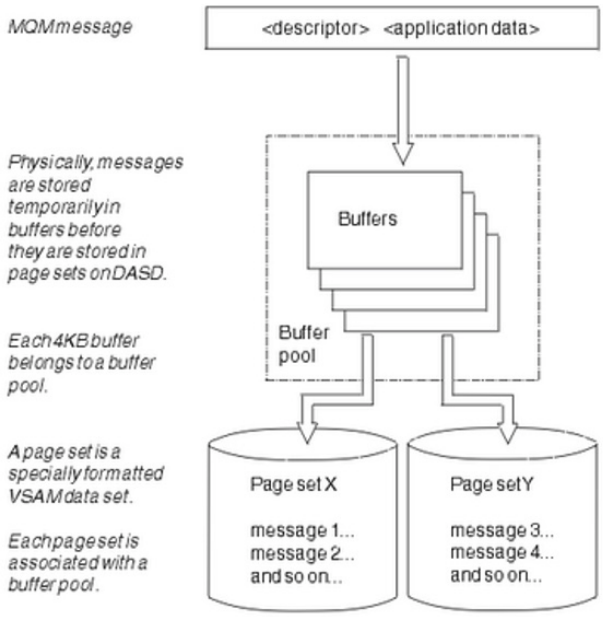


Figure 1.4: High-level IBM MQ architecture overview showing message, buffers, buffer pool, and page sets. [68]

Short-lived messages may be handled entirely in memory. However, when buffer usage exceeds certain thresholds, messages are flushed to page sets either asynchronously (via the Deferred Write Task) or synchronously (which can degrade performance). Optimizing buffer pool sizes is crucial to avoid synchronous I/O, which impacts throughput. [68]

Messaging Workflow Overview

In summary, IBM MQ enables robust, decoupled communication between applications by orchestrating message exchange through queue managers. Applications begin by connecting to a queue manager—locally or remotely—potentially using predefined channels. When an application needs to send data, it constructs a message and places it on a queue or publishes it to a topic. The target destination, whether a queue or a topic subscriber, may reside on the same queue manager or on another interconnected manager, with IBM MQ han-

dling the reliable transfer across the network. Communication occurs between queue managers, not directly between applications. [69]

IBM MQ supports different levels of message delivery assurance. Messages can be non-persistent for faster throughput or persistent to ensure durability and once-only delivery, even in the event of system or application failures. On z/OS, multiple queue managers can share access to queues via the coupling facility, enabling high-throughput parallel access. Applications may operate as IBM MQ clients or servers depending on whether they are co-located with the queue manager or not, with little impact on application logic. IBM MQ also supports both custom channel configurations and automatic channel creation, enabling scalable, high-availability deployments across distributed environments. [69]

2 Working with SMF Data in Practice

This chapter provides a practical overview of handling SMF data for analysis, covering data storage, acquisition, structure, and preparation.

It begins by briefly explaining data storage and management on mainframes, then outlines the acquisition process of SMF data from CICS and MQ subsystems using IBM utilities.

Next, the chapter addresses preprocessing methods required to transform the raw data into a human-readable format and describes the structure of SMF records, highlighting their common format consisting of metadata and data sections.

Lastly, it discusses data parsing strategies, which can be done by leveraging the previously mentioned common records format.

2.1 SMF Data Storage on Mainframes

On z/OS, data is managed in the form of data sets, which are analogous to files in distributed systems. A data set contains one or more records, which are the basic units of information processed by applications on the system. Data sets can vary in structure and access method, but two of the most common types are [19, 70] :

- **Sequential Data Sets:** In this format, records are stored one after another in a linear fashion. Accessing a specific record (e.g., the tenth record) requires reading through all preceding records, as there is no indexing mechanism.

- **Partitioned Data Sets (PDS):** A PDS is structured like a folder containing multiple named members. It includes a directory that stores the location of each member, enabling direct access. Although members themselves are stored sequentially, the presence of a directory allows for efficient retrieval and execution of specific components—such as JCL scripts, program source code, or configuration entries.

2.2 SMF Data Acquisition

High-level SMF data collection on z/OS involves subsystems writing structured monitoring information into SMF buffers. These records are then flushed into SMF data sets using IBM's IFASMFDP utility. This utility not only transfers the contents of SMF buffers to persistent storage but can also be configured to split and filter records by type and subtype, such as isolating SMF type 110 for CICS or type 115 for MQ, and copy them into dedicated sequential output datasets for further processing. [71, 20]

To facilitate transfer of the datasets, the data can be compressed using IBM's TERSE utility [72], a lossless compression tool.

2.2.1 CICS SMF 110 Data Processing

To decode SMF 110 monitoring records, two distinct steps are required [73, 74, 75] :

1. Dictionary Generation with DFHMNDUP The format of SMF 110 records can vary depending on subsystem configuration and version. Therefore, a dictionary must be generated to interpret them. This is done using the IBM utility DFHMNDUP.

2. Data Formatting with DFH$MOLS DFH$MOLS utility is used in the second stage to convert SMF 110 records into human-readable reports. It relies on both the previously created dictionary and the pre-filtered CICS data to interpret the record structure correctly. The tool cannot operate without the dictionary, as it does not inherently recognize the fields present in each record.

2.2.2 MQ SMF Data Processing

In contrast to CICS, MQ SMF records are self-defining [76]. Because of this structure, MQ does not require an external dictionary.

Data Formatting with CSQ4SMFD IBM provides a sample utility called CSQ4SMFD that can decode SMF 115 and 116 records and write

out the formatted output. The processed results are written to separate outputs for further inspection or analysis. [71, 77]

Acquiring and Formatting Datasets For This Thesis The process of obtaining and formatting the SMF datasets used in this thesis was carried out in by the thesis consultant, who handled access to the Kyndryl z/OS environment and provided this information. SMF buffers, which may contain monitoring data from various subsystems (including CICS, MQ, and others, for example type 30 or type 70 as described in Chapter 1.2.1), were first flushed into a SMF dataset. This dataset was then filtered using the IFASMFDP utility to extract the relevant SMF type 110 and 115 records. The resulting files were compressed using the TERSE utility and sent to the consultant, who decompressed and prepared them for formatting.

For CICS, the SMF 110 records were filtered again to create a dedicated dataset. Dictionaries were then generated using DFHMNDUP and passed together with the filtered CICS data to the DFH$MOLS utility, resulting in the formatted STUDENT.CICS output.

For MQ, filtering was not performed due to the small size of the dataset; instead, the complete SMF dump was passed directly into the CSQ4SMFD utility to produce the STUDENT.MQS output. While prefiltering MQ data is generally recommended to improve performance, it was deemed unnecessary in this case.

The exact formatting steps are demonstrated in the submitted JCL scripts CICS.JCL.txt and MQ.JCL.txt, which specify the program invocations and input datasets used for the data preparation process. These are included in Appendix A.3 for reference.

2.3 Records Structure

Even though IBM's SMF records, from CICS and MQ follow a similiar architectural layout, each subsystem customizes its record format to suit its own monitoring and reporting requirements. [78, 79, 76]

2.3.1 CICS Records Structure

Metadata Section

The metadata section captures general information about the record itself and the environment in which it was generated. [78, 80]

```
*  SMF HEADER          *  TIME = 10:03:33.23    DATE = 2025/049   SYSTEM-ID = LBD1    SUBSYS-ID = CICS    OPSYS = X'DE'(MVS/ESA)   *
*                      *  REC TYPE = 110   REC SUBTYPE = 1    NO OF TRIPLETS = 2                          OPSYS LEVEL = SP7.3.1    *
*                      *  PROD SECT OFFSET = 44     PROD SECT LENGTH = 114    NUM OF PROD SECTS = 1                                *
*                      *  DATA SECT OFFSET = 158    DATA SECT LENGTH = 29670  NUM OF DATA SECTS = 1                                 *
*****************************************************************************************************************************
*  SMF PRODUCT SECTION *  REC VERSION = 0730  REC MAINT IND = 0    G-APPLID = CICSD2A    S-APPLID = CICSD2A    DATA CLASS = 3      *
*                      *  JOB NAME = CICSD2A    ENTRY DATE = 2024/351  ENTRY TIME = 10:25:07.27   USER IDENTIFICATION = CICSTASK  *
*                      *  1ST CONN. OFFSET = 158    CONNECTOR LENGTH = 2      NUM OF CONNECTORS = 403                              *
*                      *  1ST DATA OFFSET  = 964    DATA ROW LENGTH  = 3608   NUM OF DATA ROWS  = 8                                *
*                      *  COMPRESSED DATA LENGTH = 5220                                                MCT OPTIONS = X'60'          *
*                      *  LOCAL TIME ZONE = FFFFD7C4  LEAP SECOND OFFSET = 00000000 00000000  DATE/TIME OFFSET = FFFFD7C4 51400000 *
*****************************************************************************************************************************
```

Figure 2.1: Structure of the Metadata Section in a CICS SMF Record

Header Captures fundamental information about the system and context in which the SMF record was created. It includes notable fields such as [78, 80] :

- **Time and Date** - Timestamp representing when the the record was moved to SMF.
- **System Identifier (SYSTEM-ID)** - Identifies the LPAR or system that created the record .
- **Record Subtype**
  - 0 – CICS journaling
  - 1 – CICS monitoring
  - 2 – CICS statistics
  - 3 – Shared temporary storage queue server statistics
  - 4 – Coupling facility data table server statistics
  - 5 – Named counter sequence number server statistics

Product Section Identifies the specific subsystem (region) that produced the monitoring data and also indicates the type or class of the record. It includes notable fields such as [78, 80] :

- **S-APPLID** and **G-APPLID** – Identifiers for the CICS region.
- **DATA CLASS** – Specifies the type of data in the record:
  - 1 : Dictionary
  - 3 : Performance
  - 4 : Exception class
  - 5 : Transaction resource class
  - 6 : Identity class
- **JOB NAME** - Identifies the z/OS job under which the region or task was executing.
- **User Identification** - Includes the user ID associated with the transaction or process.

Data Section

The data section holds the actual performance and monitoring data but varies based on the DATA CLASS. For more details refer to the section 3.1.1. [81]

Field Format and Interpretation

```
--FIELD-NAME-----------------------UNINTERPRETED-------------|-----------------INTERPRETED-------------

 DFHTASK  C001     TRAN            C3E2D4C9                          CSMI

    ...
 DFHCICS  C089     USERID          C3C9C3E2 E4E2C5D9                 CICSUSER
 DFHTASK  C004     TTYPE           E4400000                          U NULNUL
 DFHCICS  T005     START           E077A5C719B26308                2025/02/18 12:49:52.302886
 DFHCICS  T006     STOP            E077Aá5C71ADCB956                2025/02/18 12:49:52.307659
 DFHTASK  P031     TRANNUM         0041244C                           41244
 DFHTASK  A109     TRANPRI         00000001                               1
    ...
```

Figure 2.2: Segment of a performance class CICS transaction from the available dataset `STUDENT.CICS`

Each metric captured in the CICS performance class data section is composed of multiple parts that define its nature, grouping, and actual observed value. The main parts of typical entry, such as (DFHTASK C001 TRAN CSMI), can be broken down as follows [82]:

- **Group Identifier** (**DFHTASK**): Indicates the logical group or subsystem area the metric belongs to. Some common group identifiers include [83]:
  - **DFHTASK** – Task-related metrics
  - **DFHCICS** – General CICS system metrics
  - **DFHSOCK** – Socket communication metrics
  - **DFHPROG** – Program execution metrics

  For a complete list of group identifiers, refer to the official IBM documentation [83].

- **Field Identifier and Data Type** (**C001**):
  - The first character (**C**) denotes the **data type**:
    * **A** – 32-bit or 64-bit integer or string
    * **C** – Byte string (character data)
    * **P** – Packed decimal number
    * **S** – Clock value
    * **T** – Timestamp
  - The numeric portion (**001**) serves as the **field identifier**, uniquely identifying the metric within its group.

- **Informal Nickname** (**TRAN**): A human-readable label that helps quickly recognize the purpose of the field (in this case, representing a transaction identifier).

- **Observed Value** (**CSMI**): The actual value measured for this metric during the monitoring event.

2.3.2 MQ Records Structure

Header

Contains general metadata about the record, including its format, type, length, and the time and date it was created. [76, 79, 84]

Self-defining Section

Specifies the location, length, and repetition count of each data block within the SMF record. For type 115 records, this section enables accurate interpretation by indicating where each type of component data begins, how large it is, and how many instances of it are present [85]. It follows the SMF header at a defined offset from the start of the record. [76, 79, 84]

Figure 2.3 presents a segment of an SMF type 116 record. Each line represents 16 bytes of data, with each byte shown as a pair of hexadecimal digits. The left-hand column indicates the byte offset from the beginning of the record, displayed in hexadecimal format. The right-hand column displays the corresponding printable ASCII characters; non-printable bytes are represented by a dot. In this example, the self-defining section specifies that the data block begins at offset X'00000104', is X'0030' bytes long, and contains a single instance (X'0001') of the record. [76]

```
000000 01A40000 5E740035 61240100 223FD4E5  *....;.../.....MV*
000000 F4F1D4D8 F0F70000 F6F0F000 00000134  *41MQ07..600.....*
000000 00700001 00000054 00B00001 00000104  *................*
000000 00300001 00000000 00000000 00000000  *................*
000000 00000000 00000000 00000000 00000000  *................*
```

Figure 2.3: Partial hex view of an SMF 116 record, illustrating the header and the self-definig section. Source: [76]

Data Records

Contain the actual data from IBM MQ. For example, the SMF 115 records, consist of series of component-specific statistics blocks, each recorded using standardized layout. In the available dataset of this

thesis, each block includes both a hexadecimal print and a formatted field listing. [1] [79, 87]

```
--Q-P-S-T---H-E-X---P-R-I-N-T----
Address  = 234F0
00000000 : D70F0068 D8D7E2E3 00000001 00004E20 <P...QPST......+.>
00000010 : 00004E1F 00004E1F 00000001 00000000 <..+...+.........>
00000020 : 00000000 00000000 00000000 00000000 <................>
00000030 : 00000000 00000000 00000000 00000000 <................>
00000040 : 00000000 00000000 00000000 00000000 <................>
00000050 : 00000000 00000000 00000000 00000000 <................>
00000060 : 00000000 00000000                   <........        >
--Q-P-S-T---F-O-R-M-A-T-T-E-D----
qpstid   = d70f
qpstll   = 0104
qpsteyec = QPST
qpstpool = 00000001
qpstnbuf = 00020000
qpstcbsl = 00019999
qpstcbs  = 00019999
```

Figure 2.4: Segment of buffer manager statistics record from the available dataset `STUDENT.MQS`.

Hex Dump Representation The hex dump appears first in each block and displays the content of the SMF record in 16-byte rows. Each row includes the hexadecimal representation of the bytes, along with their printable ASCII equivalents. [76]

Formatted Output Below the hex print, a formatted version of the same block is shown. This section decodes the record into field–value pairs. Each field is labeled using a combination of its eyecatcher and a short descriptive metric name, such as qpstpool, which stands for the buffer pool ID [88], followed by its corresponding value 00000001.

Eyecatchers and Block identification Each statistics block includes an eyecatcher field—a 4-character identifier embedded into both the hex and formatted sections (e.g., QPST, QJST, QIS1). These eyecatchers are defined by IBM and used consistently across all record instances of a given component, making it straightforward to identify and group data by component. [85]

1. For examples of accounting records you can refer to the: [86]

2.4 Data Preparation for Analysis

2.4.1 CICS

Parsing Strategy

Since CICS data can often times come in a dumped text format [89], it is essential to parse it into table-like structure for effective analysis. The structured nature of SMF records as described, separates metadata and data sections. This enables the design of a parser to reliably identify and extract relevant parts of the raw record text. The parsing can be made by recognizing the boundaries between the metadata header, product section, and data section as defined in the CICS SMF record format. Based on field definitions provided in the dictionary record, relevant metric values can be extracted from the performance class section for each transaction. In addition, essential metadata such as system and region identifiers, timestamps, and user-related fields can be attached to each transaction entry.

While commercial tools such as the CICS Performance Analyzer [90] are available to support the entire analysis process, they are often costly and not universally accessible. Although these tools are designed for ease of use, their licensing requirements may limit availability in certain environments. That is why, in this thesis, I implemented the described parsing strategy as a custom Python-based tool to transform the CICS data into a structured, table-like format suitable for analysis. The full implementation is included as part of the submitted materials in the folder named `Smf_Cics_110_Parser`.

2.4.2 MQ

Unlike the CICS dataset, the MQ dataset used in this thesis was relatively small and reflected low system activity. As a result, manual inspection of the formatted output was sufficient, and no automated parsing logic was required. While commercial tools for MQ such as IBM Z IntelliMagic Vision for z/OS [91] can also be used to perform comprehensive analysis and visualization of MQ statistics, they are also often costly and not universally accessible.

3 SMF CICS Data Analysis

This chapter starts by describing CICS 110 records, explaining their purpose and detailing their organization into various data classes and subtypes.

It then provides an exploratory analysis of the STUDENT.CICS dataset, summarizing the time interval of data collection along with the number of regions and logical partitions (LPARs) involved, followed by an overview of the collected transactions.

Following this, it outlines structured guidelines for analyzing CICS data. These guidelines are based on my theoretical insights combined with hands-on experience with the provided dataset, and include: an entry point contribution scan, which is a multidimensional transaction scan providing a composite view for prioritizing transactions; transaction breakdown, a detailed interpretation of transaction response times, decomposed into suspend time, dispatch time, and submetrics; time-series correlation, an analysis of transaction metrics across transactions and over multiple days to identify performance trends and anomalies; and a standard follow-up checklist, which contains structured steps for translating insights into actionable optimization recommendations.

Finally, the chapter demonstrates the practical application of these guidelines through an in-depth analysis of a selected transaction, concluding with a synthesis of key findings and their implications for transaction performance optimization.

3.1 SMF CICS 110 Records

CICS generates SMF record type 110 data to track both regular transaction statistics, region-level transaction statistics, and storage-related exceptions. By analyzing these records, SMF 110 records allow detection of potential system constraints that might impact the performance of transactions. [92]

3.1.1 SMF 110 Data Classes

The records are organized into different data classes, each capturing a specific aspect of transaction behavior or system context:

- **Dictionary Class (DATA CLASS = 1)** The Dictionary Class provides the structural definitions of all possible metric fields that may appear in performance or exception records. The dictionary is generated once per region. [93]

- **Performance Class (DATA CLASS = 3)** The Performance Class captures detailed transaction-level execution data, including CPU time, elapsed time, dispatch time, and various types of suspension or wait times. This data is collected at predefined Event Monitoring Points (EMPs) embedded within the CICS code. For each transaction, one or more performance records may be generated depending on the transaction's complexity and the points it touches during its execution. [94]

- **Exception Class (DATA CLASS = 4)** The Exception Class contains monitoring data that reflects resource shortages or delays encountered during transaction execution. These may include queuing for file strings, contention for temporary storage, or other conditions that introduce latency. Such data is intended to highlight system-level constraints or inefficiencies that may degrade transaction performance. CICS generates one exception record for each individual exception condition that occurs. In addition to operational bottlenecks, exception records are also produced when CICS policy rules are triggered—specifically those that define an action such as issuing a message, logging an event, or forcing an abend. However, rules configured to reject a request do not result in the generation of an exception record. [95]

- **Transaction Resource Class (DATA CLASS = 5)** Provides additional transaction-level details about specific resources used during execution, such as files or web services. Instead of showing just totals, it breaks down usage for each individual resource. [96]

- **Identity Class (DATA CLASS = 6)** The Identity Class stores information about who started a transaction, including user identity details passed from other systems. This helps with security auditing and tracking where requests come from. [97]

3.1.2 SMF 110 Record Subtypes

SMF 110 records contains a subtype field that identifies its specific purpose. The most notable subtypes are:

- **CICS Journal Records (Subtype 0)** Managed by the CICS log manager, journal records capture chronological event data that can be used for reconstruction, auditing, or recovery. They often store information about database operations such as inserts, updates, and deletions, and can also track transaction activities over time. [98]
- **CICS Monitoring Records (Subtype 1)** These records collect runtime statistics about both user-defined and system transactions, covering aspects like execution time, CPU usage, and resource consumption. The data is essential for identifying performance issues and is commonly used in offline analysis or for chargeback purposes. [99]
- **CICS Statistics Records (Subtype 2)** Unlike monitoring records, statistics records provide information about overall system health performance and resource usage. They summarize metrics related to CICS internals, helping with long-term capacity planning and performance tuning on a global system scale. [100]

3.2 Exploratory Dataset Analysis

The STUDENT.CICS dataset used for this analysis spans across three days from approximately 12:30 on February 18, 2025 to 14:30 on February 20, 2025. Of these three days, only February 19 provides the most complete coverage, with metrics recorded consistently throughout the entire day. During these three days, data was collected from two logical partitions (LPARs): **LBD1** and **LBP1**. Within these logical partitions, five CICS regions were active: **CICSD2A**, **CICSD2F** on LBD1, and **CICSP1A**, **CICSP2A**, **CICSP2F** on LBP1.

Across these regions, a total of 12 distinct transactions were observed. Six transactions (**CRMF**, **CRSQ**, **CSOL**, **CSSY**, **CWBG**, **DTAX**) were consistently present across all regions. The remaining transactions were restricted to specific regions:

- **CDBE**, **CEX2**, and **CIST** were recorded exclusively in CICSD2A.
- **CISS** was present in both CICSD2A and CICSP2A.
- **CSMI** was observed in CICSD2A and CICSD2F.
- **CWXN** appeared across CICSD2A, CICSP1A, and CICSP2A.

The dataset consists entirely of CICS-supplied transactions, identifiable by their naming convention beginning with the letter "C". The only exception is the DTAX transaction. These built-in transactions are typically associated with CICS sample programs or are used internally by the system. In real-world performance analysis, such transactions are generally excluded, as they are not subject to optimization and may skew analytical results. However, excluding them in this case would render the dataset nearly unusable. Therefore, for the purposes of demonstration, this thesis includes these transactions as valid subjects of analysis. [101]

For a full list of CICS-supplied transactions, see [102]

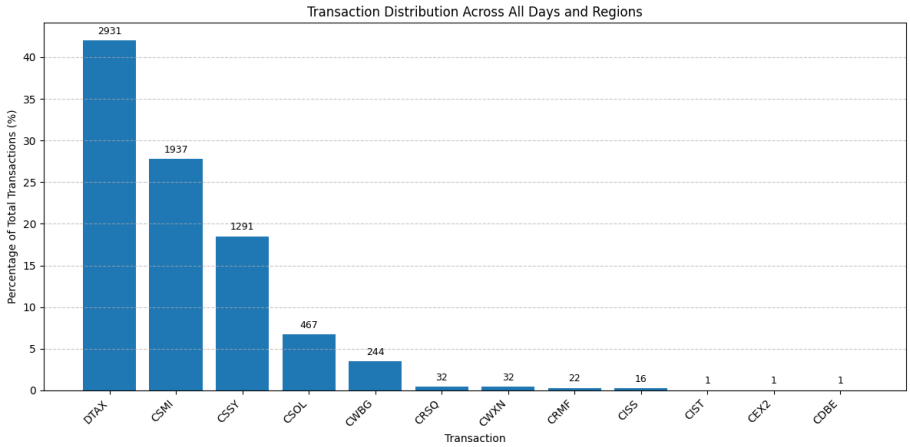


Figure 3.1: Distribution of transactions across all days and regions.

In Figure 3.1, the bar chart presents each transaction's count and its corresponding share of the total number of transactions observed. The transaction DTAX is by far the most frequent, contributing over 40% of all recorded transactions. In contrast, the transactions CDBE, CIST, and CEX2 appeared only once each. According to IBM documentation [102, 103], these transactions are associated with internal system processes such as CICS–Db2 shutdown and termination routines for TCP/IP communication links to remote systems, meaning they are system-driven maintenance or control functions rather than part of the regular transactional workload.

The dataset includes 513 defined metric fields as described in the dictionary section, along with additional relevant system-related fields from metadata sections. However, a large portion of these fields were either completely unused or sparsely populated. Specifically, 338 metrics (65.89%) had no filled values across all records, and many other metrics had a fill rate of below 50%. This is not unexpected, since inconsistencies in metric presence across transactions are inherent to the nature of CICS processing. CICS transactions can vary in complexity and behavior—some perform simple operations, while others invoke multiple programs or access different types of resources. These differences align with the transaction lifecycle described earlier in Transaction life cycle, and validate the observation that not all metrics are applicable to all transactions.

3.3 Guidelines for Analysis

The following guidelines for analyzing CICS data are derived from a combination of theoretical understanding and practical experience working directly with the provided SMF 110 dataset. They are intended to offer a structured approach to interpreting transaction-level performance characteristics in a consistent and reusable manner.

3.3.1 Entry Point Contribution Scan

One of the first steps in transaction-level performance analysis, based on my understanding of SMF CICS data, further reinforced through hands-on experience with the STUDENT.CICS dataset is identifying which transactions are most relevant for further investigation. This can be achieved by performing a comparative scan of each transaction's overall contribution to key performance dimensions such as:

- **Transaction Count Share** – The percentage of total transactions represented by each entry point.
- **Elapsed Time Share** – The proportion of cumulative elapsed time attributable to each transaction.
- **CPUT-time Share** – The share of total user CPU time spent executing logic for the transaction.
- **Suspend Time vs Dispatch Time** – The composition of each transaction's elapsed time in terms of suspend time and dispatch time.
- **CPU-to-Dispatch Ratio** – The ratio of CPU time to dispatch time.

Analyzing these dimensions together is critical. For example, a transaction might appear unimportant if it contributes only a small fraction of the overall elapsed time or if it happened only once. However, if it is frequently executed and has a disproportionately high share of CPU time or shows high suspend-to-dispatch imbalance, it may be symptomatic of underlying performance issues.

This multidimensional scan provides a composite view of transaction behavior and helps prioritize which transactions are worth

deeper inspection. It effectively acts as a weight vector for each transaction—capturing its relative significance across multiple performance dimensions. In doing so, it not only highlights transactions with unusual execution profiles, such as CPU-bound, wait-heavy, or dispatch-sensitive, but also reflects their overall contribution to system workload. This broader perspective is essential for identifying key optimization targets and shaping the direction of follow-up analysis.

3.3.2 Transaction Breakdown

Response Time Decomposition: suspend and Dispatch

Following the contribution scan, it is important to analyze the transaction's elapsed time and its internal structure to better understand the factors influencing its performance. This involves examining the two primary components of elapsed time: suspend time and dispatch time. By evaluating whether a transaction spends more time suspended or dispatched, and by calculating the CPU-to-dispatch time ratio, it becomes possible to assess how effectively system resources are being utilized.

Interpreting Key Submetrics

Within each of these categories, the most significant submetrics should be identified and interpreted in relation to the transaction's lifecycle and the nature of its execution. This step helps determine whether a transaction is primarily I/O-bound, CPU-intensive, or dependent on external systems, and whether it aligns with expected behavior based on its design and function. This provides a deeper insight into where transactions spend their time, what components they interact with, and which phases of execution may introduce delays.

Ultimately, this enables targeted performance tuning, supports informed optimization decisions, and helps uncover broader workload characteristics and system bottlenecks.

> The basic criterion of performance in a production system is response time, but what is good response time? In straightforward data-entry systems, good response time implies subsecond response time. In normal production

> systems, good response time is measured in the five to ten second range. In scientific, compute-bound systems or in print systems, good response time can be one or two minutes. Good performance, then, depends on a variety of factors including user requirements, available capacity, system reliability, and application design. Good performance for one system can be poor performance for another.

Quoted directly from IBM's performance guide for CICS [*52, Chapter 11, p. 156*].

3.3.3 Time-Series Correlation

To better understand transaction dynamics and uncover performance-related trends, it is beneficial to generate time series graphs on a per-day basis (optionally segmented by CICS region, LPAR, or workload classification). These graphs should focus on suspend time and dispatch time, along with their most relevant submetrics.

For a comprehensive list of available submetrics in SMF 110-1 records, refer to the IBM documentation [99].
The time-series correlation analysis proceeds in several steps:

- **Within a single transaction** Compare trends in suspend and dispatch time submetrics. If spikes in both occur simultaneously, this may indicate a common bottleneck.
- **Across multiple transaction types:** Identify whether different transactions experience spikes at the same time. If so, a shared system-level event is likely. If the issue is isolated to a single transaction type, it may suggest an application-specific problem.
- **Across multiple days:** Look for recurring anomalies or patterns that repeat at specific times of day or periodically. Such regularity may reflect scheduled batch jobs, interactions with external systems, or they may correspond to peak usage times.
- **Outlier detection:** Spot isolated anomalies that affect a small subset of tasks or time intervals. These may point to exceptional events, data corruption, or failed transactions. If these are significant enough, they may require deeper investigation.

Additionally, tracking the frequency, duration, and consistency of such anomalies, and overlaying these plots with total transaction counts, throughput measures, or external Key Performance Indicators can offer additional contextual insights.

3.3.4 Standard Follow up checklist

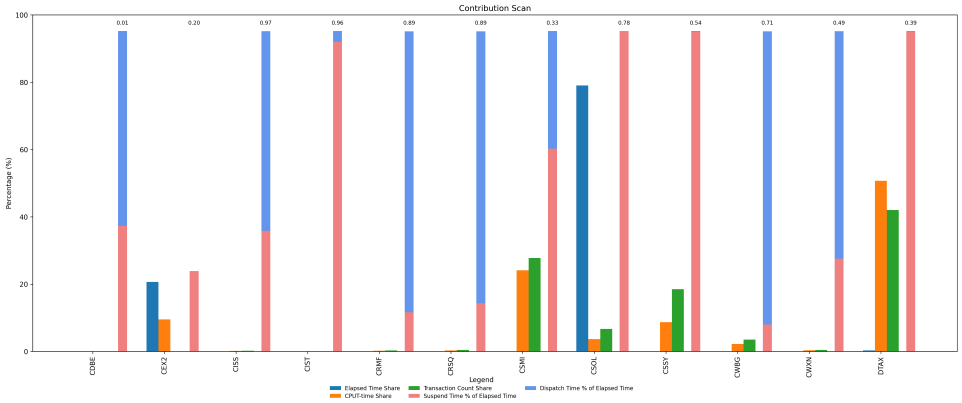
These high level follow-up actions may provide ideas that can help with translating the gathered insights into actionable optimization efforts:

- **Correlate with System-Level Metrics:** Before attributing performance issues solely to CICS-level transactions, consider broader system health. For example cross-reference CPU usage, I/O service times or paging activity.
- **Validate External Dependencies:** Examine the performance and availability of systems the transaction interacts with, such as DB2, IBM MQ, web services, or shared datasets. Look for latency, retry patterns, or contention during periods of degraded performance.
- **Inspect CICS Region Configuration:** Misconfigured regions settings or imbalanced workload distribution can lead to artificial bottlenecks. Reviewing how the regions perform can help identify configuration issues that may affect overall performance.
- **Assess Planned Workloads and Operational Windows:** Align observed anomalies with scheduled events such as batch job execution or maintenance periods. Not all performance deviations indicate faults, some may correspond with known workload spikes or operational routines.
- **Review Transaction Logic and Functional Design:** Once systemic issues are excluded, examine the transaction's logic for inefficiencies such as unnecessary I/O, excessive synchronous calls, suboptimal DB2/MQ interactions, or opportunities for asynchronous redesign.

3.4 Practical Demonstration of Guidelines

This section presents a practical application of the analytical guidelines introduced in previous chapter.

3.4.1 Contribution Scan



Note: Only compare metrics of the same type—elapsed with elapsed, CPU with CPU, etc.

Figure 3.2: Contribution scan of transactions across performance metrics.

The chart in Figure 3.2 reveals that many transactions—such as CDBE, CISS, CIST, and CWXN, register negligible values across all contribution dimensions. However, several others stand out.

- **CSOL** contributes to nearly 80% of the total elapsed time, despite having relatively low CPU time and transaction count shares. This indicates a long execution duration per instance.
- **CEX2** contributes close to 20% of total elapsed time while remaining minimal in terms of CPU and transaction frequency. Like CSOL, this suggests long-lived transactions.
- **DTAX** represents around 45% of all transaction executions and accounts for roughly 58% of total CPU time. If analysis were

based solely on elapsed time, DTAX might appear less significant. However, combining CPU utilization and frequency reveals its substantial impact on system load and processing. This illustrates how the previously mentioned weight vector can highlight transactions of interest that would otherwise be overlooked. This transaction also stands out as the only not CICS-supplied one, which in this dataset for previously mentioned reasons makes it particularly noteworthy.

- **CSMI** and **CSSY** show balanced contributions across multiple dimensions, and while not as dominant as DTAX, they may still warrant examination due to their consistent presence and moderate CPU and elapsed time shares.

3.4.2 DTAX Transaction Breakdown

As previously mentioned, DTAX is responsible for approximately 45% of all transaction volume, making it the most frequently executed transaction in the dataset. Even more notably, it accounts for approximately 58% of total CPU time consumption, despite its comparatively small share of total elapsed time. This profile suggests that while DTAX transactions complete quickly from a wall-clock perspective, they exert significant pressure on CPU resources probably due to their high execution frequency.

Breaking down DTAX's elapsed time further, 99% of it, is spent in the suspended phase, with only 1% attributed to dispatch. Within the dispatch window, the CPU-to-dispatch ratio is approximately 0.39—indicating that less than half of the dispatch phase is spent executing on the processor.

suspend Phase: The suspend interval is overwhelmingly dominated by the interval control delay time (ICDELAY [104]), which accounts for nearly 100% of the recorded suspension duration. ICDELAY measures the time a task spends waiting due to programmed delays or wait conditions defined by the application [105]. This strongly suggests that DTAX follows a timer-driven execution pattern, likely involving scheduled pauses, polling loops, or orchestrated wait intervals within its processing logic.

Dispatch Phase: DTAX's entire dispatch activity occurs under the quasi-reentrant (QR) TCB context, as indicated by QRDISPT = USRDISPT. Core CPU-related metrics—QRCPUT, USRCPUT, and CPUTONCP—also report nearly identical values, further reinforcing that all processor activity occurred directly on the QR TCB and was handled exclusively by the standard general-purpose processor.

The metric CPUTONCP reflects the time the task actively spent on a central processor, while the absence of any significant OFFLCPUT confirms that no workload was eligible for or redirected to specialty engines such as zIIP. [106]

DTAX exhibits a relatively low CPU-to-dispatch ratio of 0.39, suggesting that a substantial portion of dispatch time was consumed by non-CPU-bound phases. These may include QR TCB contention, dispatcher overhead, or waiting on internal synchronization points. Although the presence of SYNCTIME indicates that commit operations possibly to DB2 or MQ do occur, their impact on overall execution time appears minimal.

Unclassified Time: DTAX also exhibits minor activity across unclassified Resource Manager Interface (RMI) metrics, such as RMITOTAL, RMITDB2, and RMITMQM. The RMI coordinates cleanup and commit activity during task termination [107, 108]. These metrics represent backend coordination with DB2 and IBM MQ, but do not belong exclusively to either dispatch or suspend phases due to internal attribution overhead.

3.4.3 Time-Series Correlation

To analyze DTAX's runtime behavior in detail, a broad set of time-series graphs was generated for the entire three-day observation period (February 18–20, 2025). These visualizations covered key metrics and submetrics across suspend, dispatch, and unclassified RMI categories. Many of these were used to support the internal analysis, and only a select subset of representative, high-level graphs is showcased to illustrate key findings.

Recurring Patterns: A prominent finding is the presence of recurring bumps in suspend time, dispatch time, and RMI-related metrics at

consistent 6-hour intervals across all regions. This regularity strongly suggests timer-driven logic embedded in the DTAX application flow. In many cases, these spikes appeared across multiple regions and metrics simultaneously, which—when viewed at aggregated 5-minute or 30-minute intervals—give the impression of system-wide load or contention.

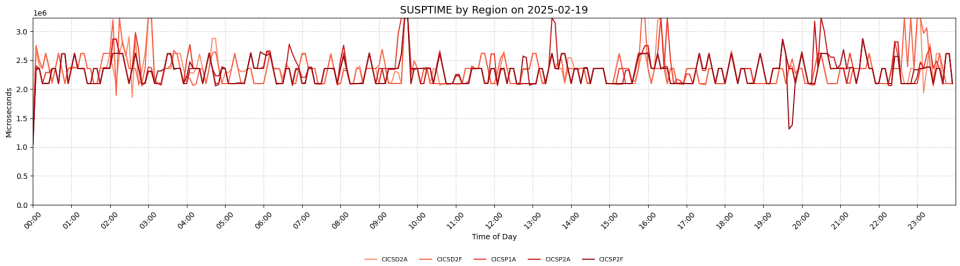


Figure 3.3: suspend time across all Regions on 2025-02-19. [1]

suspend Time Behavior: As can be seen in Figure 3.3, suspend time spikes show consistency across regions and days, with most significant rises synchronized to the 6-hour recurrence. This supports the hypothesis that DTAX is designed to execute based on pre-defined intervals rather than external input, reinforcing its classification as a timer-controlled or orchestrated task.

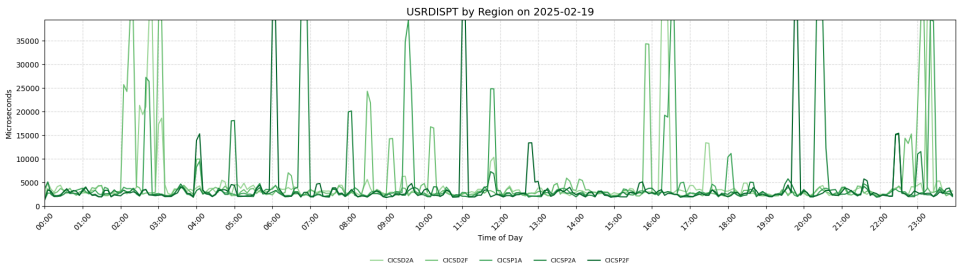


Figure 3.4: Dispatch time across all Regions on 2025-02-19. [1]

Dispatch Time Behavior: As shown in Figure 3.4, dispatch time spikes exhibit greater variability. While some align with the 6-hour periodic pattern, others appear independently. As can be seen in Figure 3.7, in some cases, spikes in dispatch time are accompanied by elevated

values in SYNCTIME—indicating that commit coordination or system-level sync operations occurred during these intervals. However, CPU-time remains relatively stable, suggesting that increases in dispatch duration are not always tied to increases in CPU processing, but may reflect queuing or QR TCB contention instead.

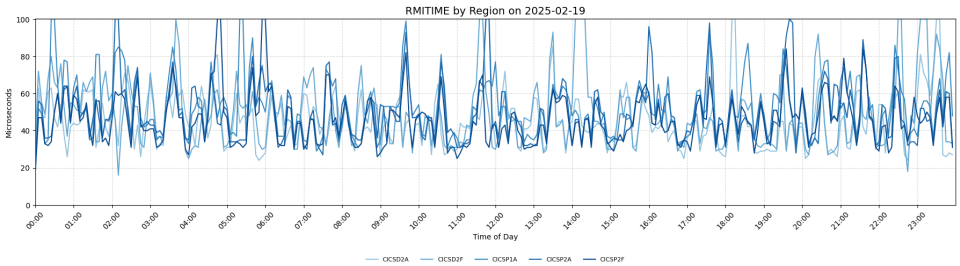


Figure 3.5: RMITIME across all Regions on 2025-02-19.

Unclassified Metrics Behavior: Visible in Figure 3.5, the RMI-related metrics also demonstrate a higher degree of volatility than suspend time metrics. While some RMI spikes loosely follow the 6-hour cadence, many deviate, likely reflecting interactions with backend systems.

Highlighted Events:

February 18

- **At around 17:00:** As can be seen in Figure 3.6, originally believed to be a system-wide contention spike, a fine-grained inspection revealed that the entire rise in both suspend and dispatch time was caused by only two overlapping DTAX transactions. The total duration of the spike was under 10 seconds, with each transaction lasting approximately 5–6 seconds. This ruled out generalized CPU load or system-wide queuing, and instead pointed to coordinated execution of timer-driven tasks.

- **At around 13:00 and 20:00:** Visible in Figure 3.7, region-specific dispatch spikes across CICSD2A and CICSP1A. The spikes at 20:00 also include increased SYNCTIME, suggesting region-local commit activity.

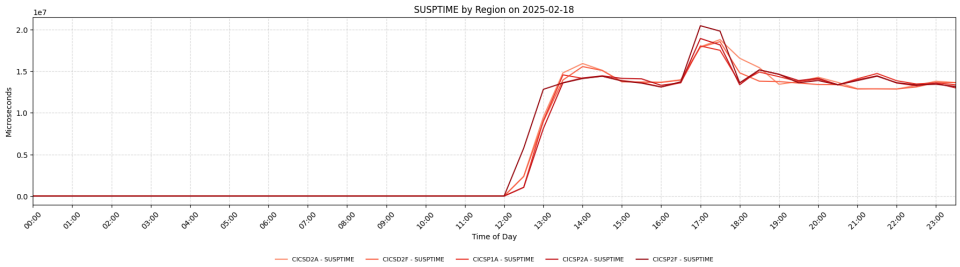


Figure 3.6: SUSPTIME across all regions on 2025-02-18. [1]

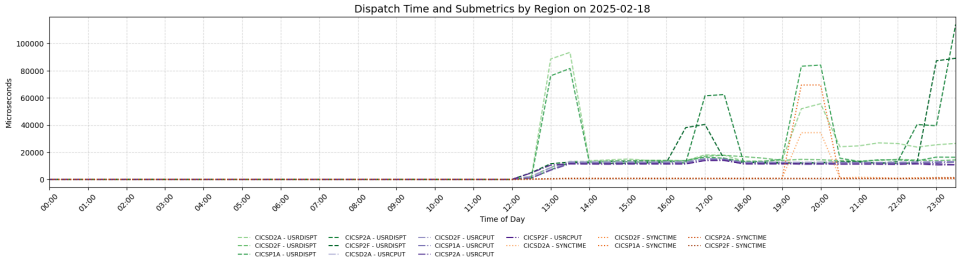


Figure 3.7: Dispatch Time supplemented with CPU-time and SYNCTIME submetrics across all regions on 2025-02-18. [1]

February 20: An extreme spike in RMITOTAL was observed. This outlier suggests a significant disruption in backend interaction, possibly caused by the unavailability of DB2 or MQ, or uncoordinated maintenance activity.

External Correlations While in the guidelines it is also recommended to correlate observed anomalies with external KPIs, throughput measures, or system-wide indicators to enrich context, the dataset used in this analysis does not include such performance metrics. As a result, comparisons were limited to concurrent transaction activity and the behavior of other transaction types. However, no strong correlations were identified between DTAX anomalies and system-wide transaction volumes during the observed intervals.

1. To improve visibility of dominant trends and submetrics, these graphs were aggregated using fixed 30-minute time intervals.

3.4.4 Conclusion of the DTAX Analysis

DTAX is a frequently executed transaction, making up nearly half of all recorded activity. While each instance is short-lived, the cumulative effect places a heavy load on system CPU resources.

Over 99% of DTAX's elapsed time is spent suspended. The interval control delays combined with recurring six-hour spikes in both suspend and dispatch time, strongly suggest a timer-driven logic pattern relying on scheduled pauses.

Initial assumptions of system-wide contention during observed spikes were invalidated by high-resolution inspection, which revealed that even the most prominent spikes originated from only a few overlapping transactions lasting less than 10 seconds in total. These spikes appeared across multiple regions and metrics simultaneously, which, when viewed at aggregated 5-minute or 30-minute intervals, gave the impression of system-wide load or contention.

All dispatch activity occurs under the quasi-reentrant TCB, with execution restricted to the standard processor. While CPU time and dispatch time generally correlate in periods of stable load, dispatch spikes often occur without a corresponding increase in CPU usage. This pattern, visible in Figure 3.7, distorts the expected CPU-to-dispatch ratio and may suggest contention or suboptimal scheduling.

RMI activity and occasional SYNCTIME values hint at backend coordination, possibly with DB2 or MQ.

Based on these findings, DTAX may benefit, if possible, from redesigning delay-driven routines into event-based triggers, optimizing backend interaction, and re-evaluating backend access patterns.

2. For this thesis, data preprocessing and visualization activities were conducted using the Python programming language, with the assistance of standard data analysis libraries: pandas[109], matplotlib[110], numpy[38], and itertools[111].

4 SMF MQ Data Analysis

This chapter begins by introducing SMF records produced by IBM MQ, specifically type 115 statistics records and type 116 accounting records. It explains their purpose in performance monitoring and outlines how these records are organized into various classes and subtypes, each capturing different aspects of MQ subsystem behavior.

It then demonstrates how to interpret and derive value from both types of records. The analysis is primarily grounded in theoretical understanding, as the insights presented, unlike those in the CICS chapter, are not backed by or derived from practical hands-on work with the available dataset. For SMF 115, the chapter showcases the analysis of component-specific statistics blocks, such as buffer manager or log manager activity. For SMF 116, it provides examples of interpreting accounting data, such as identifying inefficient queue configurations based on message counts or sizes, and assessing how resource consumption is distributed across the most active queues.

Finally, the chapter concludes with an analysis of the available MQ dataset (STUDENT.MQS). However, due to the limited size and low activity of the dataset, only minimal insights could be extracted, and no deeper analysis was feasible.

4.1 Introduction to MQ Records

IBM MQ for z/OS produces two key SMF record types: Statistics (SMF 115) and Accounting (SMF 116). Both are written to the SMF facility and provide insight into system behavior, but from different perspectives. [91]

Used together or independently, these records help monitor current usage, track trends, and support capacity planning—ultimately aiding performance tuning and cost optimization on mainframes. [112, 113]

4.1.1 SMF MQ 115 Statistics Records

IBM MQ for z/OS produces SMF type 115 records to provide periodic, system-level statistical data about queue manager activity. These records are lightweight, impose negligible CPU overhead, and are

suitable for continuous collection and long-term trend monitoring. Their main purpose is to reveal usage patterns and identify potential configuration or performance issues—such as buffer pool saturation or excessive logging activity—before they impact system behavior.

This data offers a high-level view of what is being used within the queue manager, including metrics like the frequency of MQI calls, quiet and busy periods, and resource utilization efficiency. As such, SMF 115 statistics are often the first line of analysis for diagnosing performance trends. When additional granularity is needed—such as identifying which specific applications are responsible for resource consumption—SMF 116 accounting records can be used in complement. [114, 115]

The SMF 115 records are generated by a variety of internal MQ components, each responsible for a different subsystem [116] :

- **Buffer Manager** – Oversees virtual storage buffer pools and page set I/O operations.
- **Coupling Facility Manager** – Manages the interface with the coupling facility.
- **Data Manager** – Maintains the linkage between messages and queues, invoking the buffer manager as needed.
- **Db2 Manager** – Manages the interface with the Db2 that is used as the shared repository
- **Lock Manager** – Handles resource locking across MQ processes.
- **Log Manager** – Manages log record creation for backout and recovery purposes.
- **Message Manager** – Processes API requests.
- **Storage Manager** – Manages storage. Allocates and deallocates internal memory pools or handles their expansion.
- **Topic Manager** – Maintains the topic and subscribe information.
- **SMDS Manager** – Manages Shared Message Data Sets (SMDS) within the coupling facility.

These statistics are structured into classes and subtypes, which define the scope and detail level of the collected data [22] :

- **Class 1 – Subsystem/Queue Manager Statistics:** Includes subtype 1 (system-level data), subtype 2 (messaging and queue sharing group activity), subtype 201 (page set input/output), and subtype 215 (buffer pool metrics).
- **Class 2 – Queue Manager Storage Summary Statistics:** Includes subtype 7, providing storage manager data.
- **Class 3 – Queue Manager Storage Detail Statistics:** Includes subtypes 5 and 6, containing low-level internal storage metrics (primarily used by IBM).
- **Class 4 – Channel Statistics:** Includes subtype 231, focused on the channel initiator address space.

In summary, SMF 115 statistics records offer essential insights into the general behavior of an MQ subsystem. Their broad scope and minimal system cost make them a strong foundation for ongoing monitoring. They provide a valuable baseline for proactive performance management, capacity forecasting, and minimizing operational disruptions on mainframe environments.

4.1.2 SMF MQ 116 Accounting Records

IBM MQ for z/OS produces SMF type 116 records to capture detailed accounting information about application and channel activity. These records are application-specific and provide insight into who is using what, making them an essential resource for diagnosing performance issues and profiling workload behavior. [117]

In contrast to SMF 115 statistics—which summarize resource usage at the system level—SMF 116 records offer fine-grained visibility into individual tasks. They include metrics such as CPU time per MQI call, elapsed time across internal components, total bytes processed, queue-level interactions, and delay causes (e.g., log or page set I/O). This level of detail supports a range of use cases, including chargeback accounting, performance tuning, anomaly detection, and identifying inefficient queue configurations. [117, 118, 119, 120]

Due to their granularity, these records can generate a significant volume of data and impose a non-trivial processing overhead. For this reason, they are typically enabled only periodically—for example, for a few minutes each month—to obtain representative workload samples and detect usage trends without overburdening the system. [115]

Data and Structure The content of SMF 116 records is organized by classes and subtypes, determining the level of detail [22] :

- **Class 1 – Basic Application Accounting:** Subtype 0 includes summary records with processor time per MQI call and call counts (e.g., MQPUT, MQGET).
- **Class 3 – Enhanced Task and Queue-Level Accounting:** Subtypes 1 and 2 provide extended task identification, including channel names, CPU time by queue, reasons for delays, message lifecycle timings, and byte volume.
- **Class 4 – Channel Accounting:** Subtype 10 focuses on accounting data for channel usage.

4.2 Interpreting and Deriving Value from MQ Records

Although this chapter focuses on interpreting and deriving value from MQ statistics and accounting records, it is important to note that, due to limitations in the available dataset, the content presented here, unlike in the previous CICS chapter, is based primarily on theoretical knowledge obtained from online sources.

4.2.1 SMF MQ 115 Statistics Records

The SMF 115 statistics records serve as an effective starting point for root cause analysis and trend identification when used as part of regular monitoring. They surface high-level usage information such as storage utilization, buffer pool activity, logging volume, and API call frequencies across internal MQ components. [115]

The following sections break down the most relevant MQ components for performance analysis.

DB2 Manager Statistics

In a MQ environment using shared queues, DB2 serves as the repository for object definitions and related metadata. The MQ queue manager interacts with DB2 through a pool of server tasks that execute database operations on behalf of applications. For performance monitoring, key metrics include the average server request time and the difference between wait time and server time. If these values are elevated, it may indicate the need to increase the number of server tasks. Additionally, the peak number of queued requests helps assess server load; consistently high values may suggest the need for server scaling. [118]

Coupling Facility Manager Statistics

In shared queue environments, the Coupling Facility (CF) stores messages and supports synchronized updates during commit operations. These updates may target individual messages or apply to multiple messages as a group. Single-message updates are issued as element-level modifications, while multi-message updates are grouped to maintain atomicity. If a request cannot be completed immediately, it may

be retried—these retries are recorded as redrives. To assess CF performance, one should monitor the average processing time for different request types. Additionally, any non-zero count of "structure full" events could indicate either a transient spike or ongoing capacity issues; repeated occurrences warrant investigation and possibly resizing the CF structure. [118]

Data Manager Statistics

The data manager coordinates how messages are linked to queues and relies on the buffer manager to access message content stored on disk. Its statistics reveal how frequently read-ahead operations result in disk I/O and how often messages are retrieved directly from disk. Regular monitoring of these patterns helps identify performance issues related to message access and indicates whether caching or buffer strategies may require adjustment. [118]

Message Manager Statistics

The Message Manager component handles the core MQI (Message Queue Interface) operations, including commands such as MQGET, MQPUT, and MQPUT1(a single-use variant of MQPUT used to open, put, and close in one call). Statistics from this component provide counts of these requests—both successful and failed—for each SMF interval. Monitoring these counts over time allows administrators to identify workload trends, shifting demand patterns, and potential inefficiencies. [118, 115, 114]

Tracking MQI activity supports profiling of daily or monthly peaks, identifying busy periods, and evaluating whether current capacity aligns with demand. A particularly useful diagnostic approach is comparing the relative volumes of MQGET versus MQPUT and MQPUT1 operations. As illustrated in Figure 4.1, the number of MQGET requests consistently exceeds that of puts. This pattern may reflect potential inefficiencies in message retrieval, such as multiple application instances competing for messages or the absence of indexing on queues—both of which can degrade performance. [118, 115, 114]

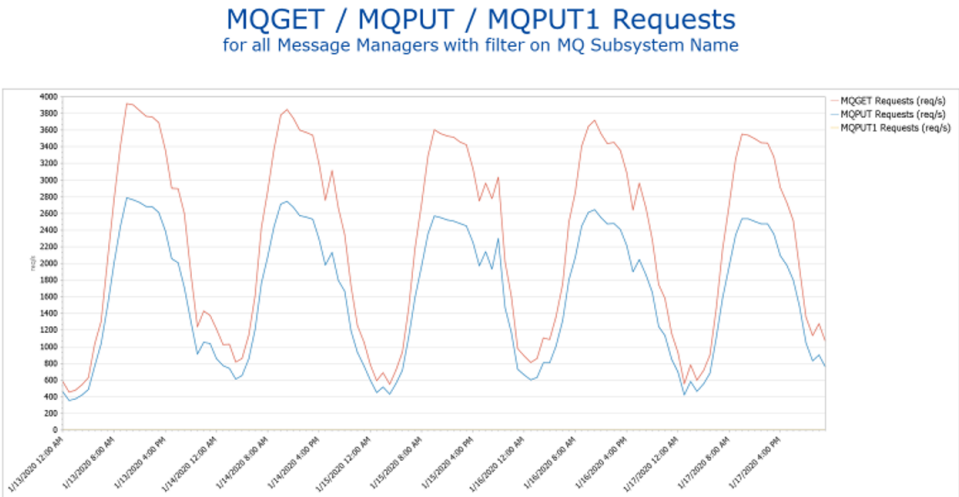


Figure 4.1: MQ request activity across time intervals. [114]

Log Manager Statistics

The Log Manager is responsible for recording all persistent data changes, such as MQPUTs and MQGETs involving persistent messages. It plays a central role in ensuring recoverability and maintaining data integrity. Logging operations write data into log buffers, which are flushed to active log datasets either during commit operations or when thresholds are met. [118]

Key indicators of logging performance include the number of times no log buffers were available, which could suggest buffer pool saturation. If this metric is high, it may signal the need to increase the number of buffers beyond the default. [119]

Logging performance may also be constrained by excessive I/O volume or high commit frequency, both of which can increase write latency. Writing more data per I/O operation is generally more efficient than performing many short writes. Conversely, frequent commits may cause inefficient I/O by forcing premature log flushes. [115]

Checkpoint frequency is another crucial metric. Checkpoints (which mark a consistent recovery point in the logs) are triggered when the active log dataset is switched or when the LOGLOAD threshold (a limit on the amount of data written to the log since the last checkpoint) is reached. Too many checkpoints can degrade performance and may

point to undersized log files or low LOGLOAD settings. Frequent application backouts can further exacerbate this issue, as rollback operations require reading the log in reverse—potentially from slower archive datasets. [121, 122, 115]

Ultimately, poor logging performance can arise from either configuration limitations such as small or small number of buffers or logs, or from application behavior—such as excessive backing out of transactions. A well-tuned system balances buffer availability, log size, checkpoint frequency, and compression effectiveness to ensure stable throughput under persistent workloads. [123]

Storage Manager Statistics

The Storage Manager handles the allocation and management of virtual memory within the queue manager. [114, 118]

One of the key benefits of monitoring Storage Manager metrics is their ability to provide early indicators of emerging storage-related issues. For instance, a non-zero count of critical Short on Storage (SOS) conditions signals a serious shortage of available memory within the queue manager. [114, 118]

Another relevant indicator is the occurrence of "Contractions" (measured both below and above the bar), which reflect MQ's internal efforts to reclaim unused memory from one subpool and reallocate it to another when moderate storage pressure arises.[114]

Buffer Manager Statistics

The Buffer Manager is a critical MQ component responsible for managing buffer pools, which in turn manage virtual storage pages. These pages are essential for supporting efficient message GET and PUT operations. Since all private queue activity requires buffer access, proper configuration of the buffer pools is vital for performance. [115]

Responsive performance in MQ depends heavily on keeping message data in memory, and buffer pools serve this role by minimizing costly synchronous I/O. As a result, properly configured and monitored buffer pools, offer one of the most effective levers for improving MQ system performance. [114]

A common misconception is that buffer pools are only relevant for persistent data. In reality, non-persistent messages are also written to page sets when insufficient buffers are available. Therefore buffer pools should be allocated and configured appropriately for both workload types. Ideally, short-lived messages should remain in memory. If buffers are exhausted, applications may block on I/O to retrieve messages from page sets. Long-lived messages, by contrast, often exceed in-memory capacity and are expected to hit disk. [115]

The configuration and tuning of buffer pools should be based on usage patterns. When one buffer pool becomes overutilized while another remains underused, queues should be reallocated to balance the load. Additionally, administrators should regularly check storage availability, because expanding memory allocation may enable further performance improvements, particularly when combined with 64-bit "above the bar" buffer pools. [119]

MQ supports buffer pools both below and above the bar. While belove-the.bar buffer pools are still used widely due to legacy reasons, above-the-bar pools, which require 64-bit addressing and sufficient memory, can allow for substantially larger buffer pools, which can significantly reduce paging and improve performance in high-throughput environments. These larger pools help keep messages in memory longer, reducing reliance on slower disk-based I/O. [124]

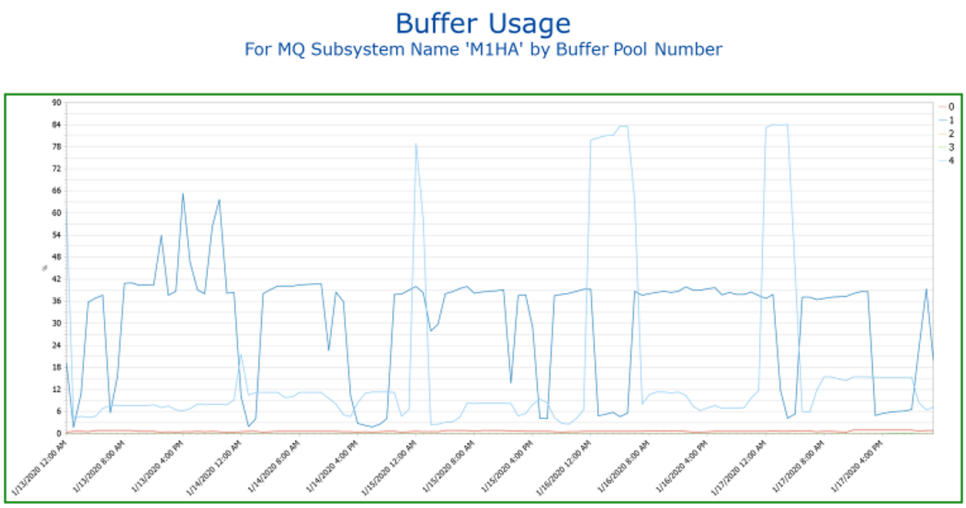


Figure 4.2: Buffer pool usage across subsystems. Source: [114]

To prevent performance degradation, MQ implements a Deferred Write Process (DWP) that activates when a buffer pool reaches 85% usage. DWP asynchronously writes the oldest updated pages to DASD to free space. If usage rises to 95%, synchronous writes occur, leading to increased application latency. Additionally, any message remaining in a buffer pool for three checkpoints is written to disk regardless of utilization. These behaviors make it crucial to monitor buffer utilization and avoid breaching these thresholds. In the example in Figure 4.2, Buffer Pool 4 repeatedly approaches the 85% threshold that triggers asynchronous destaging, potentially indicating a need to allocate additional buffers. [114]

Buffer Pool Number	Buffer Usage	DASD Read Operations	Deferred Write Threshold is Reached	Synchronous Write Threshold is Reached	Suspended - No Free Buffers
0	■	■	■	■	■
1	■	■	■	■	■
2	■	⚠	■	■	■
3	■	■	■	■	■
4	■	■	■	■	■

Figure 4.3: Health insights—drilldown by buffer pool. Source: [114]

Figure 4.3 provides a health summary for each buffer pool, indicating whether thresholds were reached or DASD reads occurred. Buffer Pool 2, for instance, shows signs of disk I/O activity that may warrant tuning. [114]

4.2.2 SMF MQ 116 Accounting Records

The detailed metrics captured in SMF 116 records enable precise workload analysis, application profiling, and infrastructure tuning. Effective interpretation of this data depends heavily on workload context

and a clear understanding of message flows and queue usage patterns. [119]

One of the most actionable insights is identifying inefficient application behavior—such as frequent open and close operations on queues—which can increase overhead and contribute to contention. Accounting records also highlight the most active queues, their associated page sets and buffer pools, and the distribution of resource consumption across them. This allows "hot spots" to be detected and evaluate buffer pool or page set balancing strategies. [115, 125]

By comparing performance across different periods—such as during normal operations versus degraded states—accounting data helps isolate root causes. For instance, increased message retrieval delays may correspond to elevated page set I/O, indicating insufficient buffer pool sizing. Such insights allow for focused adjustments rather than speculative optimization. [115]

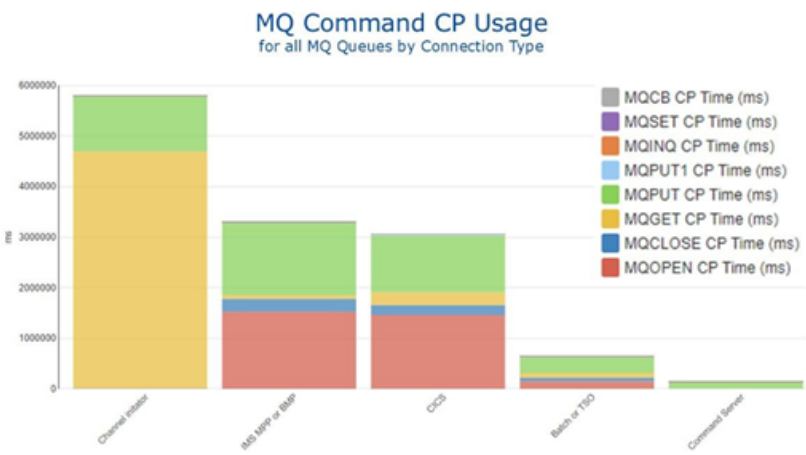


Figure 4.4: MQ CPU usage by connection type and command [126]

One particularly valuable perspective is analyzing how different connection types contribute to overall system load. As shown in Figure 4.4, detailed breakdowns of MQ command CPU time by connection type reveal the primary cost drivers—often concentrated in channel initiators and high-throughput CICS or IMS regions.[126]

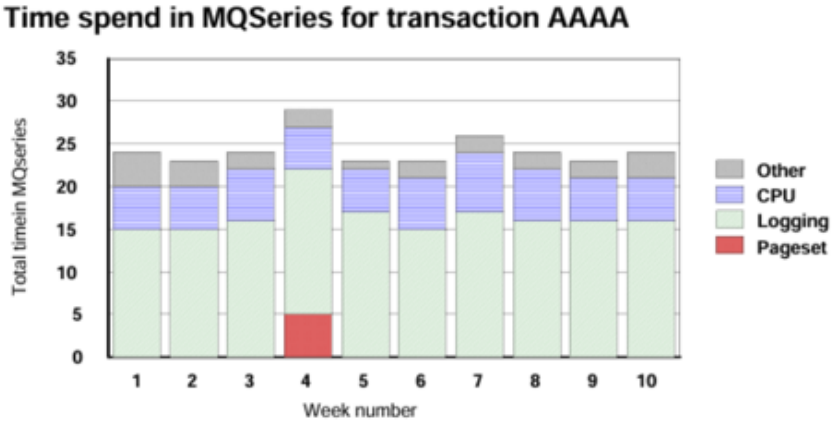


Figure 4.5: Breakdown of time spent in MQ for transaction AAAA [118]

Long-term trend analysis benefits from aggregating metrics across intervals. Weekly charts of CPU time, elapsed time, which can also be split into sub-times, and bytes processed per application can reveal workload growth, cyclical usage patterns, or the effects of recent deployments. An example of such a trend-oriented visualization is shown in Figure 4.5, which illustrates how time spent in MQ for a given transaction is distributed across categories like CPU, logging, and page set access over a 10-week span. This form of accounting-based trend analysis is particularly useful for observing gradual shifts in workload behavior and identifying recurring patterns. [118]

Simple CICS transactions typically perform one MQGET and one MQPUT1 per commit, allowing meaningful analysis using per-call averages for CPU time and elapsed time. In contrast, queue-draining transactions loop through multiple MQGETs and commits, accumulating time across calls, making cumulative metrics like total CPU or average time per commit more relevant. [118]

Further insight can be gained by drilling into correlation fields such as transaction ID, region, or subsystem—particularly in CICS workloads. These identifiers, embedded within SMF 116 records, allow detailed profiling of MQ activity. For example, the analysis of elapsed time patterns shows the spike in resource usage between 8:30 and 10:30 AM, which was traced to CICS transaction CKTI. [126]

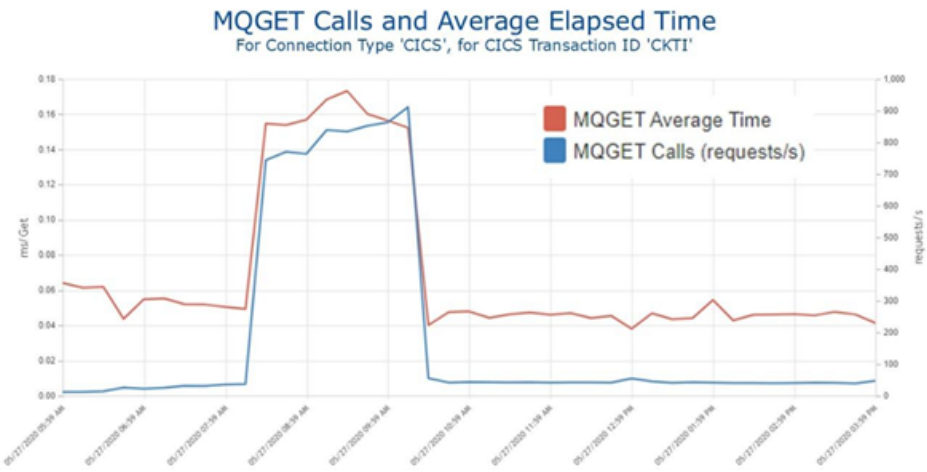


Figure 4.6: MQGET call rate vs. elapsed time for CKTI transaction [126]

In Figure 4.6, this period exhibits a strong correlation between the rate of MQGET calls and the transaction's average elapsed time, offering an example of how queue access patterns impact performance. [126]

Queue-specific metrics—such as message counts or sizes, can reveal inefficiencies in queue configurations. High rates of MQGET by MsgID or CorrelID on non-indexed queues, for example, indicate suboptimal setup. Other signs of inefficiency include excessive message expirations or repeated retrieval skips to locate matching messages. [119]

While the MQ accounting records offer rich insights, some limitations remain. For example, determining concurrency levels or linking related tasks is not straightforward. [125] Nonetheless, when used effectively, they offer a powerful insight into system behavior.

4.3 Analysis of Available Dataset

Dataset Overview The MQ dataset used in this thesis (STUDENT.MQS) consists of SMF 115 statistics records produced by various internal MQ components 4.1.1, each responsible for monitoring a specific subsystem. Since no metadata was available to describe the dataset's collection context or time range, its structure and content had to be interpreted purely from the SMF record layout. As a result, the dataset seems to reflect a single point-in-time snapshot of MQ subsystem states, rather than a sequence of measurements over time.

Component Overview Across all subsystems, the recorded metrics indicate a consistent and low-utilization environment. Activity levels appear minimal, with most components showing stable configurations and little operational pressure during the observed interval.

Scope of Analysis Due to the static nature and limited scale of the dataset, the scope of analysis was inherently constrained. Only surface-level insights could be obtained, and no deeper behavioral patterns or trends were observable.

Conclusion

This thesis addresses a critical issue in modern mainframe operations: although systems like CICS and MQ on IBM z/OS generate extensive performance data via SMF records, deriving meaningful insights from these records remains a non-trivial task. The abundance of data alone does not guarantee utility—effective analysis depends on understanding, the methodology. This work sought to bridge that gap by demonstrating how valuable insights can be extracted from SMF records and providing broad scope guidelines.

The main contribution of this thesis is a cohesive resource that combines theoretical background with practical methodology. It presents essential knowledge about mainframes, CICS, MQ, and SMF, with the transaction lifecycle and message flow descriptions further contextualizing the data analysis and enhancing understanding of system dynamics, while offering reusable analytical guidelines that can serve as learning tools or operational support. These contributions are intended to assist both newcomers and experienced professionals in gaining clearer insights into the performance characteristics of mainframe-based systems, which can then be applied in real-world operational settings. This confirms that the objective of the thesis was fulfilled.

On a personal level, this research deepened my understanding of mainframe systems and provided me an opportunity to work with not so easily obtainable SMF data. It strengthened my skills in data processing, analytical design, and interpretation of low-level system behavior. Close collaboration with an experienced consultant further enriched the experience by exposing me to insights from real-world experience and performance management practices in enterprise environments.

From a broader perspective, the thesis contributes to the field by lowering the entry barrier to SMF data analysis. It provides a roadmap for systematic performance analysis without reliance on expensive proprietary solutions, and thereby supports a more open and scalable approach to mainframe observability.

Limitations and Future work

While the analysis of CICS SMF 110 records resulted in concrete insights and guidelines grounded in both theory and practice, the same level of practical analysis could not be achieved for IBM MQ. The available dataset contained only a limited and minimally informative sample of MQ SMF 115 records, and no SMF 116 accounting records were present. As a result, the MQ-related guidelines presented in this thesis are primarily derived from theoretical knowledge and documentation rather than empirical evidence. This limitation underscores the need for broader data access in future studies.

For future work, promising directions include applying the proposed analysis guidelines to larger and more diverse datasets, including SMF record types beyond CICS and MQ. This would allow for broader performance diagnostics and richer cross-subsystem correlation. Additionally, developing open-source tools or scripts for parsing and visualizing SMF data could enhance adoption and lower the barrier to entry for newcomers, possibly making students more interested, and it would also be helpful to analysts working with mainframe systems.

Bibliography

1. VASQUEZ, Camila. *Deriving Insights from SMF 116 MQ Accounting Data*. 2024-09-27. Available also from: https://community.ibm.com/community/user/ibmz-and-linuxone/blogs/camila-vasquez2/2024/09/27/deriving-insights-from-smf-116-mq-accounting-data. Accessed: 2025-05-21.
2. IBM CORPORATION. *Mainframe: What is a mainframe? Explore how they power modern business* [online]. n.d. [visited on 2025-05-16]. Available from: https://www.ibm.com/think/topics/mainframe. Accessed 2025-05-16.
3. IBM CORPORATION. *Mainframe hardware: Processing units* [online]. n.d. [visited on 2025-05-16]. Available from: https://www.ibm.com/docs/en/zos-basic-skills?topic=concepts-mainframe-hardware-processing-units. Accessed 2025-05-16.
4. CORPORATION, IBM. *IBM z16 Technical Overview*. IBM Redbooks, 2023. Available also from: https://www.redbooks.ibm.com/abstracts/sg248493.html.
5. IBM CORPORATION. *Mainframe Hardware: I/O Connectivity* [online]. n.d. [visited on 2025-05-16]. Available from: https://www.ibm.com/docs/en/zos-basic-skills?topic=design-mainframe-hardware-io-connectivity. Accessed 2025-05-16.
6. IBM CORPORATION. *Direct Access Storage Devices (DASDs)* [online]. n.d. [visited on 2025-05-16]. Available from: https://www.ibm.com/docs/en/aix/7.2.0?topic=subsystem-direct-access-storage-devices-dasds. Accessed 2025-05-16.
7. IBM CORPORATION. *Overview of Mainframe Network Capabilities* [online]. n.d. [visited on 2025-05-16]. Available from: https://www.ibm.com/docs/en/zos-basic-skills?topic=networks-overview-mainframe-network-capabilities. Accessed 2025-05-16.

8. TECHTARGET. *Logical partition (LPAR)*. n.d. Available also from: https://www.techtarget.com/whatis/definition/logical-partition-LPAR. Accessed: 2025-05-16.

9. TECHTARGET. *z/OS* [online]. n.d. [visited on 2025-05-16]. Available from: https://www.techtarget.com/searchdatacenter/definition/z-OS. Accessed 2025-05-16.

10. IBM DOCUMENTATION. *What are mainframe operating systems?* 2024. Available also from: https://www.ibm.com/docs/en/zos-basic-skills?topic=today-what-are-mainframe-operating-systems. Accessed April 24, 2025.

11. IBM CORPORATION. *What are mainframe operating systems?* [online]. n.d. [visited on 2025-05-16]. Available from: https://www.ibm.com/docs/en/zos-basic-skills?topic=today-what-are-mainframe-operating-systems. Accessed 2025-05-16.

12. ABIS. *Subsystems in z/OS*. [N.d.]. Available also from: https://www.abis.be/html/en1374.html. Accessed: 2025-05-17.

13. BROADCOM. *Subsystems*. [N.d.]. Available also from: https://techdocs.broadcom.com/us/en/ca-mainframe-software/security/ca-auditor-for-z-os/12-1/using/technical-information/subsystems.html. Accessed: 2025-05-17.

14. IBM. *Middleware on z/OS in the 1960s*. [N.d.]. Available also from: https://www.ibm.com/docs/en/zos-basic-skills?topic=1960s-middleware-zos. Accessed: 2025-05-17.

15. WIKIPEDIA CONTRIBUTORS. *Job stream — Wikipedia, The Free Encyclopedia*. 2024. Available also from: https://en.wikipedia.org/wiki/Job_stream. Accessed: 2025-05-17.

16. IBM DOCUMENTATION. *Understanding z/OS address space*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/6.x?topic=storage-understanding-zos-address-space. Accessed: 2025-04-28.

17. IBM. *System Management Facilities (SMF) - Introduction*. [N.d.]. Available also from: https://www.ibm.com/docs/en/zos/2.4.0?topic=smf-introduction. Accessed: 2025-05-17.

18. SAHOO, Natabar. *Mainframe z/OS CPO (Capacity, Performance and Optimization) - Hints and Tips*. [N.d.]. Available also from: https://www.linkedin.com/feed/update/urn:li:activity:7061553591504801792/. Accessed: 2025-05-17.

19. BROADCOM. *CA MICS Batch and Operations Analyzer Guide*. n.d. Available also from: https://ftpdocs.broadcom.com/cadocs/0/CA%20MICS%20Resource%20Management%2012%207%2000-ENU/Bookshelf_Files/HTML/BatchandOperationsAnalyzerGuide/fd611.html. Accessed: 2025-05-12.

20. WIKIPEDIA CONTRIBUTORS. *IBM System Management Facilities*. 2025. Available also from: https://en.wikipedia.org/wiki/IBM_System_Management_Facilities. Accessed: 2025-05-13.

21. BMC SOFTWARE, INC. *SMF record type overview*. 2024. Available also from: https://docs.bmc.com/xwiki/bin/view/Mainframe/Data-for-Db2/BMC-AMI-Defender-for-Db2/amidefdb59/Getting-started/SMF-record-type-overview/. Accessed: 2025-05-17.

22. SHARKEY, Anthony. *MQ and SMF - What, When, How Much?* 2021. Available also from: https://community.ibm.com/community/user/blogs/anthony-sharkey1/2021/11/22/mq-and-smf-what-when-how-much. Accessed: 2025-05-13.

23. IBM CORPORATION. *SMF record type 30*. 2024. Available also from: https://www.ibm.com/docs/en/zos/3.1.0?topic=records-smf-record-type-30. Accessed: 2025-05-17.

24. IBM CORPORATION. *SMF type 70–79 RMF records*. 2024. Available also from: https://www.ibm.com/docs/en/zdsfcp/2.1.0?topic=performance-smf-type-70-79-rmf-records. Accessed: 2025-05-17.

25. IBM CORPORATION. *System Management Facility and Resource Measurement Facility records*. 2024. Available also from: https://www.ibm.com/docs/en/cma/2.1.0?topic=environment-system-management-facility-resource-measurement-facility-records. Accessed: 2025-05-17.

26. IBM CORPORATION. *SMF 100 Data*. 2023. Available also from: https://www.ibm.com/docs/en/z-anomaly-analytics/5.1.0?topic=classic-smf-100-data. Accessed 2025.

27. PACIFIC SYSTEMS GROUP. *SMF 100 Record Format - DB2*. 2022. Available also from: https://www.pacsys.com/smf/smf100.htm. Accessed 2025.

28. IBM CORPORATION. *SMF 101 Data*. 2023. Available also from: https://www.ibm.com/docs/en/z-logdata-analytics/5.1.0?topic=insights-smf-101-data. Accessed 2025.

29. PACIFIC SYSTEMS GROUP. *SMF 101 Record Format - DB2*. 2022. Available also from: https://www.pacsys.com/smf/smf101_DB2v10.htm. Accessed 2025.

30. IBM CORPORATION. *Type 119 SMF records*. 2024. Available also from: https://www.ibm.com/docs/en/zos/2.4.0?topic=reference-type-119-smf-records. Accessed: 2025-05-17.

31. IBM CORPORATION. *Introduction to CICS*. 2024. Available also from: https://www.ibm.com/docs/en/zos-basic-skills?topic=zos-introduction-cics. Accessed: 2025-05-17.

32. IBM CORPORATION. *Online Transaction Processing* (*OLTP*). 2024. Available also from: https://www.ibm.com/think/topics/oltp. Accessed: 2025-05-17.

33. IBM CORPORATION. *CICS in the z/OS system*. 2024. Available also from: https://www.ibm.com/docs/en/zos-basic-skills?topic=cics-in-zos-system. Accessed: 2025-05-17.

34. IBM CORPORATION. *Introduction to CICS*. 2024. Available also from: https://www.ibm.com/docs/en/zos-basic-skills?topic=zos-introduction-cics. Accessed: 2025-05-17.

35. IBM CORPORATION. *Transactions in CICS*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=applications-transactions-in-cics. Accessed: 2025-05-17.

36. IBM MAINFRAMER. *Customer Information Control System* (*CICS*) *Tutorial*. 2024. Available also from: https://www.ibmmainframer.com/cics-tutorial/customer-information-control-system/. Accessed: 2025-05-17.

37. IBM CORPORATION. *CICS: Programs, transactions, and tasks*. 2024. Available also from: https://www.ibm.com/docs/en/zos-basic-skills?topic=cics-programs-transactions-tasks. Accessed: 2025-05-17.

38. IBM CORPORATION. *CICS Transaction Processing Environment*. 2021. Available also from: https://www.ibm.com/docs/en/cics-tx/10.1.0?topic=processing-cics-transaction-environment. Accessed: 2025-05-17.

39. MICRO FOCUS. *BMS* (*Basic mapping support*). [N.d.]. Available also from: https://www.microfocus.com/documentation/enterprise-developer/ed80/ES-NET/GUID-8C4D9C6E-2F2E-479E-8AC8-8EE9162F3327.html.

40. IBM CORPORATION. *Resources and Components of a CICS Region*. 2021. Available also from: https://www.ibm.com/docs/en/cics-tx/10.1.0?topic=services-resources-components-cics-region#c_res_compnents_cics_rgn. Accessed: 2025-05-17.

41. IBM CORPORATION. *General CICS Region Services*. 2021. Available also from: https://www.ibm.com/docs/en/cics-tx/10.1.0?topic=services-general-cics-region. Accessed: 2025-05-17.

42. WIKIPEDIA CONTRIBUTORS. *CICS — Customer Information Control System*. 2024. Available also from: https://en.wikipedia.org/wiki/CICS. Accessed: 2025-05-17.

43. IBM CORPORATION. *Roles of a CICS region in a topology*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/5.5.0?topic=topology-roles-cics-region-in-cicsplex. Accessed: 2025-05-17.

44. IBM CORPORATION. *What does the dispatcher do?* 2024. Available also from: https://www.ibm.com/docs/en/zos-basic-skills?topic=system-what-does-dispatcher-do. Accessed: 2025-05-17.

45. IBM CORPORATION. *Dispatchable units of work: Tasks and service requests*. 2024. Available also from: https://www.ibm.com/docs/en/zos-basic-skills?topic=system-dispatchable-units-work-tasks-service-requests. Accessed: 2025-05-17.

46. IBM CORPORATION. *Introduction to task control blocks (TCBs)*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/5.5.0?topic=path-introduction-task-control-blocks-tcbs. Accessed: 2025-05-17.

47. IBM CORPORATION. *Quasi-reentrant task control block (QR TCB)*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/5.5.0?topic=path-quasi-reentrant-task-control-block-qr-tcb. Accessed: 2025-05-17.

48. IBM MAINFRAMER. *CICS Environment*. 2024. Available also from: https://www.ibmmainframer.com/cics-tutorial/cics-environment/. Accessed: 2025-05-17.

49. IBM CORPORATION. *CICS Transaction Server for z/OS V5.6: Glossary of Terms*. 2020. Available also from: https://publibfp.dhe.ibm.com/epubs/pdf/dfhg600.pdf. Accessed: 2025-05-17.

50. TUTORIALSPOINT. *CICS - Nucleus*. 2024. Available also from: https://www.tutorialspoint.com/cics/cics_nucleus.htm. Accessed: 2025-05-17.

51. IBM. *CICS Transaction Life Cycle*. 2024. Available also from: https://www.ibm.com/docs/en/cics-tx/10.1.0?topic=processing-life-cycle-cics-transaction. Accessed: 2025-04-28.

52. IBM CORPORATION. *CICS Performance Guide*. 2001. Available also from: https://ftpmirror.your.org/pub/misc/ftp.software.ibm.com/software/cics/pdf/dfht300.pdf. Accessed: 2025-05-17.

53. IBM CORPORATION. *IBM CICS Transaction Server for z/OS 5.5 Monitoring Reference*. 2018. Available also from: https://www.ibm.com/docs/en/SSGMCP_5.5.0/pdf/monitoring-reference_pdf.pdf. Accessed: 2025-05-17.

54. GROUP, Virtual CICS User. *Problem Analysis and Performance Tuning for CICS*. n.d. Available also from: https://virtualusergroups.com/wp-content/uploads/presentations/Virtual%20CICS%20-%20Problem%20Analysis%20and%20Performance%20Tuning%20for%20CICS.pdf. Accessed: 2025-05-17.

55. IBM DEVELOPER. *IBM MQ Fundamentals*. 2023. Available also from: https://developer.ibm.com/learningpaths/ibm-mq-badge/mq-fundamentals/. Accessed: 2025-05-01.

56. IBM CORPORATION. *IBM MQ 9.2: Message Persistence*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=messages-message-persistence. Accessed: 2025-05-17.

57. IBM CORPORATION. *WebSphere Application Server 9.0.5: MQ Messages*. 2024. Available also from: https://www.ibm.com/docs/en/was-nd/9.0.5?topic=provider-mq-messages. Accessed: 2025-05-17.

58. IBM CORPORATION. *IBM MQ 9.2: Messages and Queues*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=queuing-messages-queues. Accessed: 2025-05-17.

59. IBM CORPORATION. *IBM MQ 9.4: Types of Messages*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.4.x?topic=messages-types-message. Accessed: 2025-05-17.

60. IBM CORPORATION. *IBM MQ 9.4: Introduction to Message Queuing*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.4.x?topic=overview-introduction-message-queuing. Accessed: 2025-05-17.

61. IBM CORPORATION. *IBM MQ Queues* (*Version 9.2.x*). [N.d.]. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=explorer-mq-queues. Accessed: 2025-05-10.

62. IBM DEVELOPER. *Message and Queue Fundamentals*. 2022. Available also from: https://developer.ibm.com/articles/awb-message-and-queue-fundamentals/. Accessed: 2025-05-17.

63. IBM CORPORATION. *IBM MQ 9.2: Types of MQ Queue Managers*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=types-mq-queue-managers. Accessed: 2025-05-17.

64. IBM DOCUMENTATION. *The Queue Manager on z/OS*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.3.x?topic=concepts-queue-manager-zos. Accessed: 2025-05-01.

65. HARRISON, Howard. *Chapter 15: WebSphere MQ* [PowerPoint presentation]. 2014. Available also from: https://www.slideserve.com/howardharrison/chapter-15-websphere-mq-powerpoint-ppt-presentation. Accessed: 2025-05-01.

66. IBM CORPORATION. *IBM MQ Channels* (*Version 9.2.x*). [N.d.]. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=explorer-channels. Accessed: 2025-05-10.

67. ITGIX. *MQ: Introduction to Message-Oriented Middleware and IBM MQ*. 2022. Available also from: https://itgix.com/blog/mq-introduction-to-message-oriented-middleware-and-ibm-mq/. Accessed: 2025-05-10.

68. SHARKEY, Anthony. *MQ for z/OS: Page Set Performance and Best Practice*. 2022. Available also from: https://community.ibm.com/community/user/blogs/anthony-sharkey1/2022/02/08/mq-for-zos-page-set-performance-and-best-practice. Accessed: 2025-05-01.

69. IBM CORPORATION. *Introduction to IBM MQ*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.4.x?topic=mq-introduction. Accessed: 2025-05-17.

70. IBM. *What is a data set?* 2023. Available also from: https://www.ibm.com/docs/en/zos-basic-skills?topic=more-what-is-data-set. Accessed: 2025-05-12.

71. JOHNSTON, Neil. *MQ SMF - Everything You Need to Know*. 2018. Available also from: https://www.mqtechconference.com/sessions_v2018/MQ_SMF.pdf. Accessed: 2025-05-13.

72. WIKIPEDIA CONTRIBUTORS. *Terse*. 2025. Available also from: https://en.wikipedia.org/wiki/Terse. Accessed: 2025-05-13.

73. IBM CORPORATION. *Monitoring dictionary utility program DFHMNDUP*. 2023. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=utilities-monitoring-dictionary-utility-program-dfhmndup. Accessed: 2025-05-13.

74. IBM CORPORATION. *Sample monitoring data print program DFHMOLS*. 2023. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=utilities-sample-monitoring-data-print-program-dfhmols. Accessed: 2025-05-13.

75. IBM CORPORATION. *Processing CICS monitoring data*. 2023. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=dfhmols-processing-cics-monitoring-data. Accessed: 2025-05-13.

76. IBM DOCUMENTATION. *Layout of SMF Type 116 Record*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=data-layout-smf-type-116-record#q038420___q038420_1. Accessed: 2025-05-10.

77. IBM DOCUMENTATION. *Processing Type 115 SMF Records*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=statistics-processing-type-115-smf-records. Accessed: 2025-05-20.

78. IBM CORPORATION. *CICS SMF record formats - CICS TS 6.x*. 2023. Available also from: https://www.ibm.com/docs/en/cics-ts/6.x?topic=data-cics-smf-record-formats. Accessed: 2025-05-13.

79. IBM CORPORATION. *IBM MQ Version 9.4: Layout of SMF type 115 record*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.4.x?topic=statistics-layout-smf-type-115-record. Accessed: 2025-05-17.

80. IBM CORPORATION. *SMF header and SMF product section - CICS TS 6.x*. 2023. Available also from: https://www.ibm.com/docs/en/cics-ts/6.x?topic=data-smf-header-smf-product-section. Accessed: 2025-05-13.

81. IBM DOCUMENTATION. *CICS Section in SMF Records*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/6.x?topic=data-cics-section. Accessed: 2025-05-20.

82. IBM CORPORATION. *CICS data monitoring fields - CICS TS 6.x*. 2023. Available also from: https://www.ibm.com/docs/en/cics-ts/6.x?topic=monitoring-data-fields-cics-data. Accessed: 2025-05-13.

83. IBM CORPORATION. *Performance Class Data Listing (Data Fields)*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/6.x?topic=fields-performance-class-data-listing-data. Accessed: 2025-04-28.

84. IBM CORPORATION. *SMF Header - MQ Statistics*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.4.x?topic=statistics-smf-header. Accessed: 2025-05-13.

85. IBM CORPORATION. *Self-defining sections - MQ Statistics*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.4.x?topic=statistics-self-defining-sections. Accessed: 2025-05-13.

86. CRESS, Morag Hughson. *Introduction to SMF 116 Records*. 2013. Available also from: https://www.mqtechconference.com/sessions_v2013/Intro2SMF116.pdf. Accessed: 2025-05-17.

87. IBM CORPORATION. *Examples of SMF Records - MQ Statistics*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.4.x?topic=statistics-examples-smf-records. Accessed: 2025-05-13.

88. IBM CORPORATION. *SMF115 BP record type - IBM Z Log Data Analytics*. 2024. Available also from: https://www.ibm.com/docs/en/z-logdata-analytics/5.1.0?topic=data-smf115-bp-record-type. Accessed: 2025-05-13.

89. IBM DOCUMENTATION. *Formatting and Interpreting Dumps*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/5.5.0?topic=determination-formatting-interpreting-dumps. Accessed: 2025-05-20.

90. IBM DOCUMENTATION. *CICS Performance Analyzer*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=workload-cics-performance-analyzer. Accessed: 2025-05-20.

91. IBM. *IntelliMagic Vision for z/OS – IBM MQ Support*. 2024. Available also from: https://www.ibm.com/products/z-intellimagic-vision-for-zos/mq. Accessed: 2025-05-20.

92. IBM CORPORATION. *SMF 110 Data*. 2023. Available also from: https://www.ibm.com/docs/en/z-logdata-analytics/5.1.0?topic=insights-smf-110-data. Accessed 2025.

93. IBM CORPORATION. *Dictionary and data sections - CICS TS 5.6*. 2020. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=section-dictionary-data-sections. Accessed: 2025-05-13.

94. IBM CORPORATION. *Performance class - CICS TS 5.6*. 2020. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=data-performance-class. Accessed: 2025-05-13.

95. IBM CORPORATION. *Exception class data*. 2020. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=data-exception-class. Accessed: 2025-04-28.

96. IBM CORPORATION. *Transaction and resource class - CICS TS 5.6*. 2020. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=data-transaction-resource-class. Accessed: 2025-05-13.

97. IBM CORPORATION. *Identity class - CICS TS 5.6*. 2020. Available also from: https://www.ibm.com/docs/en/cics-ts/5.6.0?topic=data-identity-class. Accessed: 2025-05-13.

98. PACIFIC SYSTEMS GROUP. *CICS Journal Records (SMF 110 Subtype 0)*. Available also from: https://www.pacsys.com/smf/smf110_subtype0.htm. Accessed: 2025-05-20.

99. PACIFIC SYSTEMS GROUP. *CICS Monitoring Records (SMF 110 Subtype 1)*. Available also from: https://www.pacsys.com/smf/smf110_subtype1.htm. Accessed: 2025-05-20.

100. PACIFIC SYSTEMS GROUP. *CICS Statistics Records (SMF 110 Subtype 2)*. Available also from: https://www.pacsys.com/smf/smf110_subtype2.htm. Accessed: 2025-05-20.

101. IBM CORPORATION. *CICS/VS MVS Version 2.1.2: CICS-Supplied Transactions*. 1991. Available also from: https://bitsavers.org/pdf/ibm/370/CICS_MVS/SC33-0511-1_CICS_MVS_Version_2.1.2_CICS-Supplied_Transactions_Mar1991.pdf. SC33-0511-1.

102. IBM CORPORATION. *CICS-supplied transactions*. 2024. Available also from: https://www.ibm.com/docs/en/cics-ts/6.1?topic=reference-cics-supplied-transactions. Accessed: 2025-04-30.

103. IBM. *IPCONN resource definitions*. 2025. Available also from: https://www.ibm.com/docs/en/cics-ts/6.x?topic=attributes-ipconn-resource-definitions. Accessed: 2025-05-17.

104. BMC SOFTWARE, INC. *SMF 110 Fields of the Transaction Monitoring Performance Class*. 2024. Available also from: https://docs.bmc.com/xwiki/bin/view/Mainframe/Security/BMC-AMI-Datastream-for-zOS/datastrmzos71/Command-and-syntax-reference/Supported-record-field-names/SMF-110-fields/SMF-110-fields-of-the-transaction-monitoring-performance-class/. Accessed: 2025-05-17.

105. ZETALY DOCUMENTATION. *SMF 110-1 – CICS TS for z/OS Statistics*. 2024. Available also from: https://docs.zetaly.io/docs/smf-110-1-cics-ts-for-zos-statistics. Accessed: 2025-05-17.

106. IBM. *Relating CICS Transactions to Hardware Resources*. 2023. Available also from: https://www.ibm.com/docs/en/cics-ts/5.5.0?topic=planning-relating-cics-transactions-hardware-resources. Accessed: 2025-05-17.

107. IBM. *Using Resource Managers*. 2020. Available also from: https://www.ibm.com/docs/en/zos/2.4.0?topic=topics-using-resource-managers. Accessed: 2025-05-17.

108. BROADCOM INC. *RMI – Resource Manager Interface*. 2020. Available also from: https://techdocs.broadcom.com/us/en/ca-mainframe-software/database-management/smart-restart-and-smart-rrsaf/20-1/smart_restart_reference/profile-parameters-defined-at-installation-time/rmi----resource-manager-interface.html. Accessed: 2025-05-17.

109. PANDAS DEVELOPMENT TEAM. *pandas—Python Data Analysis Library*. n.d. Available also from: https://pandas.pydata.org/. Retrieved May 20, 2025.

110. MATPLOTLIB DEVELOPMENT TEAM. *Matplotlib—Visualization with Python*. n.d. Available also from: https://matplotlib.org/. Retrieved May 20, 2025.

111. PYTHON SOFTWARE FOUNDATION. *itertools—Functions creating iterators for efficient looping*. n.d. Available also from: https://docs.python.org/3/library/itertools.html. Retrieved May 20, 2025.

112. IBM DOCUMENTATION. *IBM CICS TS Monitoring Reference*. 2024. Available also from: https://www.ibm.com/docs/en/SSJL4D_6.x/pdf/monitoring-reference_pdf.pdf. Accessed: 2025-05-20.

113. IBM DOCUMENTATION. *Using the System Management Facility with IBM MQ for z/OS*. 2024. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=zos-using-system-management-facility. Accessed: 2025-05-20.

114. VASQUEZ, Camila. *Learning from SMF MQ Statistics*. 2025. Available also from: https://community.ibm.com/community/user/ibmz-and-linuxone/blogs/camila-vasquez2/2025/03/19/learning-from-smf-mq-statistics?communityKey=1d107e16-3c1e-45f1-ac77-e53129e79ff5. Accessed: 2025-05-14.

115. UNKNOWN. *MQ203: Performance Accounting*. 2014. Available also from: https://www.slideshare.net/slideshow/201407-mq203-performance-accounting/36629536. Accessed: 2025-05-14.

116. IBM CORPORATION. *Interpreting IBM MQ for z/OS Performance Statistics*. 2023. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=zos-interpreting-mq-performance-statistics. Accessed: 2025-05-13.

117. MQGEM SOFTWARE. *MQ PCF Accounting and Statistics*. n.d. Available also from: https://mqgem.wordpress.com/mq-pcf-acct-stat/. Accessed: 2025-05-14.

118. IBM CORPORATION. *SupportPac MP1B: MQ Accounting and Statistics Report Generator*. 2023. Available also from: https://ftpmirror.your.org/pub/misc/ftp.software.ibm.com/software/integration/support/supportpacs/individual/mp1bv7.pdf. Figure source: mq_transaction_time_spent.png, Accessed: 2025-05-01.

119. IBM CORPORATION. *The Dark Arts of MQ SMF Evaluation*. 2017. Available also from: https://share.confex.com/share/123/webprogram/Handout/Session16202/The%20Dark%20Arts%20of%20MQ%20SMF%20Evaluation2.pdf. Accessed: 2025-05-14.

120. IBM CORPORATION. *Investigating MQ Performance Problems*. 2022. Available also from: https://www.ibm.com/docs/en/ibm-mq/9.2.x?topic=zos-investigating-performance-problems. Accessed: 2025-05-14.

121. IBM CORPORATION. *IBM WebSphere MQ V7.0: System Administration Guide*. 2009. Available also from: https://public.dhe.ibm.com/software/integration/wmq/docs/V7.0/PDFs/V7.0_2009/csqsat05.pdf. Accessed: 2025-05-14.

122. IBM CORPORATION. *IBM MQ V9.1: Troubleshooting Guide*. 2022. Available also from: https://public.dhe.ibm.com/software/integration/wmq/docs/V9.1/PDFs/mq91.troubleshoot.pdf. Accessed: 2025-05-14.

123. IBM CORPORATION. *IBM MQ V8.0 Planning Guide*. 2014. Available also from: https://public.dhe.ibm.com/software/integration/wmq/docs/V8.0/PDFs/mq80.plan.pdf. Accessed: 2025-05-20.

124. REDBOOKS, IBM. *IBM MQ for z/OS: Performance and Capacity Planning*. IBM, 2016. Available also from: https://www.redbooks.ibm.com/redbooks/pdfs/sg248218.pdf. Chapter 12: Advantages of a buffer pool above the bar. Accessed: 2025-05-14.

125. IBM CORPORATION. *Session 11383 – SMF 115 and 116 Performance Records*. 2014. Available also from: https://share.confex.com/share/119/webprogram/Handout/Session11383/Session%2011383%20-%20SMF115%20and%20116.pdf. Accessed: 2025-05-14.

126. VASQUEZ, Camila. *Learning from SMF MQ Accounting - MQGET Calls and Elapsed Time for CKTI Transaction*. 2025. Available also from: https://community.ibm.com/community/user/ibmz-and-linuxone/blogs/camila-vasquez2/2025/03/19/learning-from-smf-mq-accounting?communityKey=1d107e16-3c1e-45f1-ac77-e53129e79ff5#:~:text=Figure%208%20%2D%20MQGET%20Calls%20and%20Average%20Elapsed%20Time%20for%20CICS%20Transaction%20CKTI%20(%C2%A9%20IntelliMagic%20Vision). Figure source: mq_gets_avg_elapse_tran.png, Accessed: 2025-05-01.

A An appendix

A.1 SMF CICS 110 Parser Overview

The folder `Smf_Cics_110_Parser`, included with this thesis, contains the implementation of a custom parser for CICS SMF 110 records. The parser was developed in Python to extract structured performance data from raw CICS records based on known record layouts.

The tool is designed to separate the record into its logical components—namely the header, product section, and data section—while distinguishing between dictionary records and performance records. It utilizes modular extractor components to obtain relevant metrics by parsing the necessary fields from each section.

The main entry point of the parser is the file `main.py`, and the resulting output is exported in a tabular format suitable for analysis using Python libraries such as `pandas`. The implementation also makes use of several standard Python libraries, including `pathlib`, `typing`, `enum`, `re`, and `abc`, to handle filesystem operations, type annotations, structured enums, pattern matching, and abstract interfaces respectively.

A.2 Formatted Datasets Used in Analysis

The formatted datasets used in this thesis are derived from SMF records collected from a Kyndryl z/OS system. These files were produced by applying the formatting utilities described in Section A.3.

- `STUDENT.CICS` – contains transaction performance data from multiple CICS regions, formatted via `DFH$MOLS`.
- `STUDENT.MQS` – contains IBM MQ statistics records, formatted via `CSQ4SMFD`.

Both datasets are included in the digital archive accompanying this thesis.

A.3 JCL Scripts for SMF Data Formatting

This thesis includes two Job Control Language (JCL) scripts used for formatting SMF records on the mainframe system. These scripts provide the necessary control statements for executing the IBM utilities `DFH$MOLS` and `CSQ4SMFD`, which convert SMF data into readable output.

- `CICS.JCL.txt` – used to format SMF 110 records into the dataset `STUDENT.CICS`.
- `MQ.JCL.txt` – used to format SMF 115/116 records into the dataset `STUDENT.MQS`.

The scripts are included in the submitted materials for reference and reproducibility.

A.4 Figures Folder

All figures referenced in the thesis are included in the `Figures` folder. They are provided to allow the reader to examine the visualizations in full detail.